



University of
Nottingham

UK | CHINA | MALAYSIA

Deep Learning for Hyperspectral Image Classification in Remote Sensing

Submitted April 27, 2026, in partial fulfillment of
the conditions for the award of the degree **BSc Computer Science with Artificial Intelligence**.

Run Hu
20513301

Supervised by Kian Ming Lim

School of Computer Science University of Nottingham Ningbo China

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Dr. Kian Ming Lim. I feel incredibly lucky to work with him. He was always there whenever I ran into problems, and I am especially grateful for how quickly he always got back to my emails with such detailed explanations. His constant support and timely advice made the whole research process much smoother and more enjoyable than I ever expected.

Abstract

Hyperspectral image (HSI) classification deals with images and presents challenges due to high dimensionality, mixed pixels, and noise. Unlike traditional image classification tasks where each image is associated with one class, in HSI classification, each pixel within one single image is labeled with a specific class. This leads to complex data with high spectral and spatial variability, which makes accurate classification difficult. Additionally, the high dimensionality increases computational complexity while mixed pixels and noise can further degrade classification performance. To address these challenges, this research proposes a hybrid model Dual-Branch Convolution with Pixelwise Vision Transformer (DBCP). The DBCP model adopts a dual-path strategy to synergize local spectral-spatial information extraction with global contextual modeling. The CNN branch uses a 3D-CNN for non-linear dimensionality reduction, followed by a Separate Groupwise Convolution (SGC) block. The SGC block employs multi-scale kernels to capture spectral-spatial features. On the other hand, the Pixelwise Vision Transformer (ViT) branch treats individual pixels as patches, which uses a multi-head self-attention mechanism to model long-range spatial dependencies. By aggregating information from surrounding pixels, the ViT branch provides robust contextual cues to resolve the classification of mixed pixels at class boundaries. Furthermore, a Squeeze-and-Excitation (SE) block is integrated to recalibrate channel importance, allowing the network to focus on the most relevant spectral-spatial information. Experimental results show that the proposed DBCP model outperforms state-of-the-art methods across all benchmark datasets in this study, including Indian Pines, Botswana, Salinas, and WHU-Hi-LongKou. Specifically, it achieves over 99% Overall Accuracy (OA) on both the Salinas and Botswana datasets, while also delivering strong results on Indian Pines and WHU-Hi-LongKou, with OA values of 97.23% and 98.68%, respectively.

Contents

Acknowledgments	i
Abstract	ii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Aims and Objectives	3
2 Literature Review	4
2.1 CNN-based Methods	4
2.2 ViT-based Methods	5
2.3 Hybrid Methods	6
2.4 Mamba-based Methods	9
2.5 Brief Summary	10
3 Methodology	11
3.1 Patch Segmentation	11
3.2 CNN Branch	12
3.2.1 Channel Layer Normalization	12
3.2.2 Pointwise Convolution (PWC)	13
3.2.3 Dimensionality Reduction with 3D-CNN	13
3.2.4 Spectral Grouped Convolution (SGC) Block	14
3.2.5 Squeeze-and-Excitation (SE) Block	16
3.3 ViT Branch	17
3.3.1 Dimensionality Reduction with PCA	17
3.3.2 Global Context Capture with Pixelwise ViT	18
3.4 Feature Fusion and Classification	19
4 Experimental Result and Analysis	21
4.1 Experiment Setting	21
4.2 Performance Comparison	29
4.2.1 Evaluation on Indian Pines Dataset	29
4.2.2 Evaluation on Botswana Dataset	29
4.2.3 Evaluation on Salinas Dataset	30
4.2.4 Evaluation on WHU-Hi-LongKou Dataset	30
4.3 Ablation Study	31
4.4 Computational Efficiency Analysis	32
4.5 Brief Summary	32
5 Reflection	33
5.1 Project Management	33
5.2 Risk Management	35
5.3 Model Evolution	35
5.4 Reflection on original plan	37
6 Conclusion	38
6.1 Future Work	39

References	40
Appendices	45
A Example: Source Code Samples	45

Chapter 1

Introduction

1.1 Background and Motivation

Hyperspectral Images (HSI) typically involves two spatial dimensions and one spectral dimension. The spatial resolution describes the geometric arrangement of the image pixels, while the spectral resolution refers to how the illumination within the pixels varies with wavelength. These data are often represented as a 3D hyperspectral data cube, where each “slice” along the spectral axis corresponds to a specific band from the electromagnetic spectrum [1].

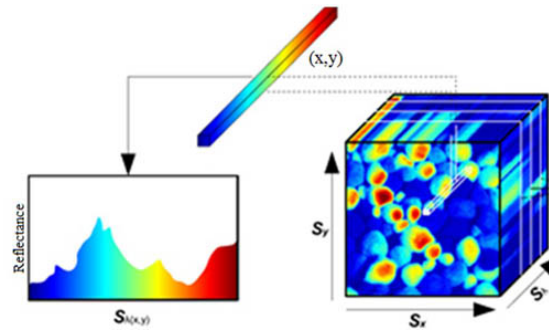


Figure 1.1: an example of HSI [2]

HSI plays a crucial role in analyzing data from various earth observation systems and it has wide applications in different areas, including:

- Water Resources Control [3]
- Food Control [4]
- Agricultural Applications [5]
- Medical Diagnosis [6]
- Crime Scene Details Analysis [7]
- Land Cover Identification [8]
- Urban Change Detection [9]
- Military [10]
- Environment Monitoring [11]

Therefore, an efficient HSI classification model is crucial and valuable because of its wide applications in remote sensing and significant benefits to society.

With the emergence of models like CNN and ViT, the accuracy of HSI classification has been greatly improved. Compared to traditional machine learning methods such as SVM and Random Forest, deep learning models offer significant advantages in HSI classification by overcoming the limitations of hand-crafted feature engineering [12]. Traditional approaches often rely on heuristic descriptors that struggle to capture the inherently nonlinear relationships between captured spectral information and corresponding materials. Furthermore, while traditional pixelwise classifiers focus on spectral signatures and neglect spatial contexts, deep learning architectures like 3D-CNN enable the seamless integration of spatial-spectral features. Consequently, the transition toward deep network has become the main trend by providing better performance in complex remote sensing scenarios.

HSI contain rich spatial and spectral information, but also present significant challenges in classification due to their high dimensionality, noise, mixed pixels, and the limited availability of labeled samples [13]. Additionally, deep neural network might suffer from feature dilution. Consequently, to facilitate the development of more robust classification frameworks, the following five key challenges are summarized:

- **High-Dimensionality:** Hyperspectral image has very strong inter-band correlation and there is often a large amount of redundant information in the image, and such information needs to be removed before any further processing can be carried out. [14] Additionally, the high dimensionality leads to an increased number of parameters, making the classification tasks more complex. Without appropriate dimensionality reduction strategies, the training process is found to be exceptionally time consuming, often requiring dozens of hours to complete even on modern hardware.
- **Mixed Pixels (Spectral Mixing):** Hyperspectral image classification is complicated by the coexistence of pure and mixed pixels [15]. While traditional full-pixel models assume a one-to-one mapping between a pixel and a material, mixed-pixel models account for the spectral overlap of multiple materials, which means a single pixel sometimes covers multiple classes or materials, leading to ambiguous labels, especially near boundaries. The spectral signatures of mixed pixels are often found to be inconsistent with the standard profiles of any single class, leading to high intra-class variance and inter-class similarity. These mixed pixels frequently result in misclassification when relying solely on spectral information.
- **Limited Training Samples and Inter-class Imbalance:** Unlike traditional computer vision domain, which usually contain thousands of classes, hyperspectral images have far fewer classes (around 10 classes) to recognize. Therefore, hyperspectral image classification might not require a huge amount of data for training [16]. However, it remains a challenge to extract useful spectral-spatial information under limited training samples. Moreover, in hyperspectral image classification, labeled data are often imbalanced across classes, making supervised training more difficult and unstable. Specifically, this imbalance creates a biased feature space where the network tends to favor majority classes, leading to a significant drop in the recognition accuracy of minority categories.
- **Vanishing Gradients and Feature Dilution:** As the network depth increases such as CNN architectures, the vanishing gradient problem becomes more obvious, which makes

training more time consuming and resource intensive [17]. Additionally, in the interim report model, which use a serial CNN-ViT connection, the ViT module is set to process features filtered by the CNN layers. This serial combination leads to the information loss of original spectral-spatial details and prevents the ViT from leveraging the full context of the raw data.

- **Noisy Labels:** It is often challenging and impractical to collect clean labels for training samples because the labels may be corrupted [18]. However, this challenge of noisy labels is excluded from the primary scope of this work because label noise is a limitation in sensor technology and data collection logistics in remote sensing rather than a fundamental flaw in spectral-spatial feature representation.

1.2 Aims and Objectives

The primary goal of this study is to propose a novel HSI classification model that outperforms existing models (in terms of OA, AA, and Kappa) on multiple HSI benchmark datasets, while maintaining computational efficiency. To tackle the challenges identified above and fulfill this goal, the following objectives have been outlined correspondingly:

1. Develop an information-preserving spectral reduction strategy by grouping spectral bands into learnable subsets and processing them through parallel branches to control parameter growth, thereby substantially reducing training time and model complexity while maintaining the integrity of discriminative spectral features.
2. Develop a method for effectively modeling the relationships among neighboring pixels to improve the classification of mixed pixels, particularly in boundary and spectrally ambiguous regions.
3. Design a framework capable of learning robustly from limited labeled samples by incorporating efficient feature recalibration modules such as the Squeeze-and-Excitation (SE) block.
4. Propose a dual-branch architecture that preserves spectral-spatial information and alleviates the information loss that may occur in conventional single-stream deep networks.
5. Conduct a comprehensive performance evaluation of the proposed model on benchmark hyperspectral image datasets using standard metrics such as OA, AA, and Kappa, and to verify the contribution of individual components through ablation studies.

Chapter 2

Literature Review

Extensive research has been conducted on current advanced Hyperspectral Image (HSI) classification models. The researched models and their explanations are presented below:

2.1 CNN-based Methods

3D-CNN [19] is a pioneering deep learning model that explores spectral-spatial feature extraction. It simultaneously capture discriminative information from both spectral and spatial dimensions. To mitigate the risk of overfitting caused by limited training samples, several regularization strategies such as L2 regularization, ReLU, and Dropout are integrated into the training process. Furthermore, a virtual sample enhanced method is proposed to expand the training set by simulating radiation disturbances and mixture variations. This model provides a solid foundation for deep learning in HSI classification.

HybridSN [20] is a CNN-based model that integrates both 3D-CNN and 2D-CNN. First, Principal Component Analysis (PCA) is applied for dimensionality reduction. Then, three layers of 3D-CNN are used to capture spectral-spatial information. Afterward, one layer of 2D-CNN is applied to capture spatial information. Finally, the output is passed through fully connected layers for classification. This model is relatively simple yet highly efficient.

CNN-PPF [21] is a CNN-based model that uses deep pixel-pair features. First, a pixel-pair model is constructed by pairing the center pixel with its neighbors to quadratically increase the number of training samples. Then, a deep CNN architecture with ten convolutional layers is employed to learn highly discriminative features from these pairs. During testing, a voting strategy is applied to determine the final label based on the joint classification results of multiple pixel-pairs.

Hybrid 3D-2D CNN [22] also combines both 3D-CNN and 2D-CNN layers to efficiently extract spectral and spatial features. To mitigate the computationally expensive issue caused by 3D-CNN, the model introduces depthwise separable convolution in the 3D block, which reduces the computational cost significantly. After the 3D-CNN layers, the output is passed through 2D-CNN layers to further extract spatial features, making the model more robust. This model is quite similar with HybridSN.

SPRN [23] combines spectral partitioning with residual learning. Instead of processing the full spectral cube directly, the model first divides the input spectral bands into several continuous sub-bands. Then, grouped convolutions are used to implement parallel feature extraction from different spectral groups. To enhance information exchange across sub-bands, the model

introduces an improved residual block consisting of a local branch and a global branch. In addition, a Homogeneous Pixel Detection Module (HPDM) is proposed to emphasize pixels that are spectrally similar to the center pixel and suppresses interfering pixels.

S3Net [24] uses a lightweight spatial-spectral network (SSN), composed of 1D-CNN and 2D-CNN to extract spectral-spatial features. Based on two SSN branches, a Siamese framework is constructed to augment the training process through sample pairing. In addition, two input patches with different sizes are fed into the two branches to provide complementary features. This model is lightweight and effective for hyperspectral image classification under few-shot settings.

However, CNN-based model typically requires a large number of parameters to be trained, which is why PCA is commonly used for dimensionality reduction. In contrast, grouped convolution and 3D-CNN-based dimensionality reduction can also reduce computational complexity without relying on PCA, thereby preserving more spectral information.

2.2 ViT-based Methods

SpectralFormer [25] addresses the limitations of traditional CNNs and RNNs by focusing on spectral information rather than spatial features. The model introduces two key modules. First is Groupwise Spectral Embedding, which learns from neighboring spectral bands to capture local spectral relationships. Another is Cross-Layer Adaptive Fusion, which uses a skip connection to merge features from different layers to reduce information loss. These innovations allow SpectralFormer to effectively handle both local spectral details and long-range dependencies across layers.

WaveFormer [26] is a transformer-based model that combines wavelet transforms with a Vision Transformer. It uses wavelet-based convolution to downsample data while preserving important spatial and spectral information, which avoids the information loss of average pooling. In addition, WaveFormer extracts multiscale spatial-spectral features and processes them using the transformer’s attention mechanism to capture both local and global relationships.

Group-Aware Hierarchical Transformer (GAHT) [27] addresses issues such as the over-dispersion of features in vision transformers. The model integrates a Grouped Pixel Embedding module that divided the spectral features into non-overlapping subchannels, which enhances the capture of local spatial-spectral relationships while maintaining the global context through the Multi-Head Self-Attention mechanism. This method ensures that the model attends to both local and global dependencies within the spectral and spatial dimensions. Furthermore, GAHT employs a hierarchical transformer architecture that progressively reduces the dimensionality of features in each stage, making the model more computationally efficient while improving classification accuracy.

Masked Vision Transformer [28] introduces three key designs. First is Blockwise Patch Embedding, which uses different linear projections for different spectral intervals to better capture the diversity of hyperspectral signals across wavelength ranges. Second is Spatial-Spectral Factorization, which decomposes self-attention into spatial self-attention and spectral self-attention, thereby reducing computational complexity while preserving both spatial context and spectral

dependencies. Finally, masked self-supervised pre-training is applied by masking a portion of spatial-spectral patches and reconstructing the masked pixel values. These designs allow the model to learn more suitable spectral-spatial representations for hyperspectral data.

DSFormer [29] performs dual selective fusion of spatial-spectral information. The network first applies PCA and shallow convolution to generate low-level features, and then employs two Dual Selective Fusion Transformer Groups for deep feature extraction. Each group contains a Kernel Selective Fusion Transformer Block (KSFTB), which adaptively fuses multiscale features to learn an appropriate receptive field, and several Token Selective Fusion Transformer Blocks (TSFTBs), which select the most relevant spatial-spectral tokens for self-attention computation. By jointly selecting suitable receptive fields and informative tokens, DSFormer reduces redundant information interference and improves spatial-spectral context modeling, leading to superior classification performance on multiple benchmark hyperspectral datasets.

HSI-BERT [30] is a Transformer-based model that leverages bidirectional encoder representations for hyperspectral image. First, the spectral bands of each pixel are treated as a sequence of tokens, similar to words in a sentence. Then, a multi-layer bidirectional Transformer encoder is employed to capture long-range dependencies and global contextual information across the entire spectral range. Afterward, a self-attention mechanism is used to adaptively weigh the importance of different bands for feature representation. Finally, the learned global spectral features are fed into an output layer for classification. This model demonstrates the superiority of attention-based architectures over traditional sequence models in handling high-dimensional HSI data.

MSTNet [31] is a multilevel spectral-spatial Transformer network designed for comprehensive feature representation. First, a multilevel architecture is employed to capture hierarchical features from different depths of the HSI data. Then, spectral and spatial Transformer modules are integrated at each level to capture global spectral dependencies and spatial contextual information, respectively. Afterward, features from multiple levels are fused through skip connections to enhance the discriminative power and ensure stable gradient flow. Finally, the aggregated multilevel features are passed through fully connected layers for final classification. This model effectively leverages both local details and global relationships across different scales.

However, ViT models typically require a large amount of data for training while HSI classification is often performed with a small amount of data. In this case, the pixelwise approach ensures that there are sufficient parameters for training, enabling the model to capture long-range dependencies.

2.3 Hybrid Methods

The Groupwise Separable Convolutional Vision Transformer (GSC-ViT) [32] proposes a Groupwise Separable Convolution module combining Grouped Pointwise Convolution and Grouped Convolution to efficiently extract local spectral-spatial features while significantly reducing the number of parameters. After feature extraction, the model uses a lightweight Vision Transformer block, which employs Groupwise Separable Multihead Self-Attention (GSSA) to capture both local and global spatial dependencies. GSSA separates the traditional multihead self-attention mechanism into Groupwise Self-Attention for local feature extraction and Pointwise

Self-Attention for global feature extraction.

Hybrid Conv-ViT Network (HCVN) [33] also combines CNN and Vision Transformer. It features a dual-branch architecture: one branch extracts spatial features using hybrid convolution and ViT, while the other branch captures spectral features with 3D-CNN. This structure allows the model to integrate spatial and spectral information effectively. By processing the entire image rather than patches, HCVN captures the global context and avoids the loss of local structure seen in traditional ViT models.

Center-Masked Transformer [34] integrates two key strategies. The first is the Local-to-Global Token Embedding framework, which enhances feature extraction by dividing the embedding task into subtasks that focus on individual spectral subbands, and then combining the local features to generate global tokens. This approach captures better spatial-spectral information while reducing computational complexity. The second strategy is the Regularized Center-Masked Pretraining (RCPT), a self-supervised pretraining method that enables the model to learn dependencies between central and neighboring pixels using unlabeled data. The RCPT task includes two subtasks: sample reconstruction and center pixel reconstruction, which improves the model's ability to generalize and enhances its classification accuracy.

Double Branch Convolution-Transformer (DBCT) [35] integrates a 3D-CNN branch to extract local spectral and spatial features and a Convolutional Transformer Encoder branch to capture global spectral and spatial information. The model introduces a Multiscale Spectral Feature Extraction module to enhance spectral feature representation by extracting features at multiple scales. This dual-branch structure allows DBCT to efficiently model both local and global features. It provides a lightweight and effective solution for HSI classification with limited training data.

CTMixer model [36] integrates both CNN and transformer to capture both local and global features as well. The model consists of three main components: a Group Parallel Residual Block (GPRB) to extract spectral-spatial features, a Transformer Encoder with Convolution (TEC) branch, and a CNN branch. The GPRB helps capture local spectral-spatial features through grouped convolutions, while the TEC branch focuses on global spectral features using transformer-based attention mechanisms. The Local-Global Multihead Self-Attention mechanism further improves performance by incorporating convolution operations within the attention mechanism. The output features from both branches are fused to provide an accurate classification so that this hybrid approach allows the model to efficiently capture both detailed local features and global dependencies.

Spectral-Spatial Feature Tokenization Transformer (SSFTT) [37] is designed to efficiently capture both spectral and spatial features. The model first uses a spectral-spatial feature extraction module, which combines 3D-CNN and 2D-CNN to capture shallow spectral and spatial features. These features are then transformed using a Gaussian-weighted feature tokenizer, which generates semantic tokens by mapping the feature maps into a new semantic space. These tokens are fed into the transformer encoder module and the multi-head self-attention captured long-range dependencies across spectral and spatial features, which enhances the model's ability to learn deep semantic representations. Finally, the output tokens are classified using a linear layer and softmax function. This approach efficiently reduces computational complexity while maintaining high classification accuracy.

Local Semantic Feature Aggregation-based Transformer (LSFAT) [38] combines CNN-based feature extraction with a transformer architecture to effectively capture both local and global features. The model uses a 3D-CNN module for shallow feature extraction, followed by the Neighborhood Aggregation-based Embedding and Neighborhood Aggregation-based Attention modules to capture multiscale spatial-semantic features. These modules help merge pixel-wise feature tokens from homogeneous regions, which preserves fine-grained information while learning large-scale structures. The transformer encoder then processes these tokens and incorporates a reusable classification token to capture long-range dependencies to facilitate efficient classification.

MorphFormer [39] incorporates learnable morphological operations into the transformer pipeline. Instead of relying only on standard patch embeddings and self-attention, the model first applies spatial and spectral morphological convolutions, including dilation and erosion operations, to enhance the structural characteristics of the patch tokens. In this way, the network can better capture not only spectral responses and spatial context, but also shape, boundary, and texture information of ground objects. The morphologically enriched patch tokens are then fused with the CLS token and processed through multihead attention, allowing the model to learn more discriminative spectral-spatial representations for hyperspectral image classification.

CAEVT [40] combines a 3D convolutional autoencoder with a MobileViT block. The 3D convolutional autoencoder is first used for nonlinear dimensionality reduction, allowing the network to preserve more original spectral-spatial information than traditional linear methods while reducing spectral redundancy. After that, the extracted features are fed into a MobileViT module, which integrates convolution and transformer operations to learn both local and global representations. In this way, the model can simultaneously capture fine-grained neighborhood information and long-range dependencies, leading to more discriminative feature representations for hyperspectral image classification. This design enables CAEVT to achieve strong classification performance while maintaining a relatively lightweight architecture.

MASSFormer [41] combines a shallow CNN with a memory-augmented Transformer encoder. A shallow CNN composed of 3D and 2D convolution blocks is used to capture joint spectral-spatial features. To preserve useful local context and salient information, a Memory Tokenizer (MT) is introduced to convert the shallow features into memory tokens through average pooling and max pooling. These memory tokens are then incorporated into a Memory-Augmented Transformer Encoder (MATE), where they are used to enhance the key and value branches of self-attention for more sufficient spatial-spectral information mixing. Finally, the classification token is passed to an MLP head for prediction.

DATN [42] is a CNN-Transformer hybrid model. It adopts a four-stage hierarchical framework in which each stage contains an SE-Conv block and a spatial-spectral hybrid transformer (SSHT) module. The SE-Conv block performs channel adjustment and channel attention, while the SSHT module combines spectral self-attention and spatial self-attention to capture global spatial-spectral dependencies and local spatial information. In addition, a spectral local-convolutional block (SLCB) is introduced in parallel through skip connections to preserve local spectral information that may be weakened by the transformer branch. Finally, the global and local spatial-spectral features are fused and fed into a linear classifier for prediction.

SSAN [43] is an attention-based deep learning framework. First, a series of convolutional layers are utilized for initial feature extraction. Then, a spectral attention module is integrated

to recalibrate the importance of various bands, emphasizing discriminative spectral signatures. Simultaneously, a spatial attention module is employed to focus on informative spatial structures within the neighborhood. Finally, these refined spectral and spatial features are fused and passed through fully connected layers for classification.

A^2S^2K -ResNet [44] is a deep residual network that incorporates an attention-based adaptive kernel mechanism. First, the model employs a spectral-spatial attention module to recalibrate the importance of spectral bands and spatial regions. Then, it uses an adaptive kernel selection strategy, allowing the network to dynamically adjust the size of convolution kernels for multi-scale feature extraction. Afterward, residual blocks are implemented to facilitate deep feature learning while preventing gradient degradation. Finally, the refined features are fused for the classification of land cover classes.

3DOC-SSAN [45] combines 3D Octave Convolution (3D OctConv) with a spatial-spectral attention mechanism. First, the 3D OctConv is employed to factorize the HSI features into high and low frequency components, which effectively reduces spatial redundancy and computational cost. Then, the spatial-spectral attention network (SSAN) is integrated to adaptively refine the features by emphasizing informative bands and representative spatial regions. Afterward, a residual learning structure is used to facilitate the extraction of deep hierarchical features. Finally, the optimized features are fed into fully connected layers for land cover classification.

2.4 Mamba-based Methods

SpectralMamba [46] combines a state space model with deep learning techniques to address challenges like spectral variability and confusion in hyperspectral data. The model uses Piecewise Sequential Scanning to break down the spectral data into manageable pieces, which improves computational efficiency while maintaining essential spectral features. Additionally, the Gated Spatial-Spectral Merging module enhances the model’s ability to capture both spatial and spectral relationships by adaptively merging these features. This enables SpectralMamba to focus on the most important spectral bands without relying on expensive attention mechanisms or recurrent models, offering a more efficient solution for large-scale HSI tasks.

IGroupSS-Mamba [47] focuses on improving computational efficiency while extracting both spatial and spectral information. It leverages an Interval Group S6 mechanism, which divides high-dimensional spectral data into non-overlapping groups to reduce redundancy and computational load. Each group is processed using a unidirectional state-space model for sequence modeling with different scanning directions applied across spectral and spatial dimensions. While the model’s core component is the Interval Group Spatial-Spectral Block, which combined spatial and spectral operators to capture global spatial-spectral relationships. The architecture is hierarchical and enables multiscale feature extraction with a pixel aggregation strategy helped with efficient downsampling.

3DSS-Mamba [48] integrates state-space models with a 3D spectral–spatial selective scanning mechanism to effectively capture global spectral–spatial dependencies. It first converts the hyperspectral data into spectral–spatial tokens using a spectral–spatial token generation module. Then, the 3DSS mechanism performs selective scanning on the 3D hyperspectral tokens, which facilitates pixelwise interactions across both spectral and spatial dimensions using five

different scanning routes. The model captures intricate global relationships by leveraging 3D spectral–spatial Mamba blocks that sequentially processed these tokens for semantic feature extraction. The final features are passed through a classifier to make predictions. This framework achieves improved performance with lower computational costs compared to traditional methods like CNNs and Transformers, making it efficient for high-dimensional hyperspectral data.

2.5 Brief Summary

Mamba is a relatively new architecture and there are fewer experiments using Mamba-based models for HSI classification tasks. In contrast, CNN and ViT are well-established architectures with a longer history and extensive research, so there is a lot of papers available for review. While CNN are considered simple and effective for HSI classification, long-range dependencies are often difficult to capture using this architecture. Consequently, the ViT is used to address this limitation. Based on all those reasons, a hybrid model combining CNN and ViT architectures has been developed.

Chapter 3

Methodology

A hybrid model **DBCP** is proposed and the model structure is described as the Figure 3.1. The proposed model consists of two main branches: a CNN branch and a ViT branch. As shown in the model structure, the input is first divided into patches through patch segmentation, after which the two branches are used to extract features for classification.

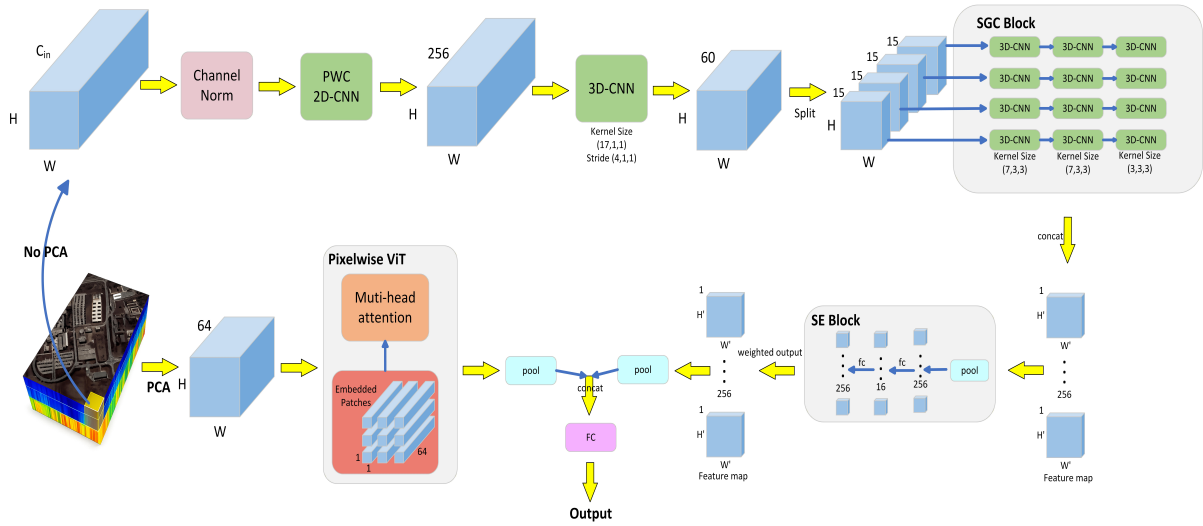


Figure 3.1: Model Structure

3.1 Patch Segmentation

The patches are generated using a sliding-window strategy. Specifically, a fixed-size window is slid over the hyperspectral image, and a local patch is extracted at each valid center position. This allows the model to construct patch-based samples that preserve both the spectral signature of the center pixel and the local spatial information of its surrounding neighborhood.

Given a hyperspectral image

$$X \in \mathbb{R}^{H \times W \times B} \quad (1)$$

and its corresponding label map

$$Y \in \mathbb{R}^{H \times W}, \quad (2)$$

a square patch of size

$$P \times P \quad (3)$$

is extracted around each labeled pixel. Therefore, each training sample can be represented as

$$\mathcal{P}_{x,y} \in \mathbb{R}^{P \times P \times B}, \quad (4)$$

where (x, y) denotes the center pixel location and the class label of the patch is assigned according to the label of the center pixel.

To ensure that pixels near the image boundary can also be used as patch centers, zero-padding is applied to the hyperspectral image and label map before patch extraction. Specifically, if the patch size is P , the padding width is set to

$$p = \left\lfloor \frac{P}{2} \right\rfloor \quad (5)$$

For each valid pixel position (x, y) , a local patch centered at that pixel is cropped from both the original hyperspectral data and the PCA-reduced data. In this way, the network can use the local neighborhood of the center pixel to learn spectral-spatial representations, rather than relying only on the spectral signature of a single pixel.

3.2 CNN Branch

3.2.1 Channel Layer Normalization

Given an input feature tensor

$$X \in \mathbb{R}^{B \times C \times H \times W}, \quad (6)$$

where B denotes the batch size, C is the number of channels (spectral bands), and H and W are the spatial height and width, channel layer normalization performs normalization along the channel dimension at each spatial location.

For each sample b and spatial position (h, w) , the channelwise mean is computed as

$$\mu_{b,h,w} = \frac{1}{C} \sum_{c=1}^C X_{b,c,h,w}, \quad (7)$$

and the corresponding variance is

$$\sigma_{b,h,w}^2 = \frac{1}{C} \sum_{c=1}^C (X_{b,c,h,w} - \mu_{b,h,w})^2 \quad (8)$$

Then, the normalized feature is obtained by

$$\hat{X}_{b,c,h,w} = \frac{X_{b,c,h,w} - \mu_{b,h,w}}{\sqrt{\sigma_{b,h,w}^2 + \varepsilon}}, \quad (9)$$

where ε is a small constant added for numerical stability.

After normalization, a learnable affine transformation is applied:

$$Y_{b,c,h,w} = \gamma_c \hat{X}_{b,c,h,w} + \beta_c, \quad (10)$$

where γ_c and β_c denote the learnable scaling and bias parameters for the c -th channel, respectively.

In tensor form, the whole process can be written as

$$Y = \gamma \odot \frac{X - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta, \quad (11)$$

where $\odot$ denotes element-wise multiplication with broadcasting over the batch and spatial dimensions.

Unlike standard Layer Normalization over all feature dimensions, the proposed formulation only normalizes across channels at each spatial position, which helps preserve spatial structure while reducing inter-channel distribution variation.

3.2.2 Pointwise Convolution (PWC)

The pointwise convolution (PWC) enables the model to project input features from different hyperspectral datasets into a unified spectral dimension. In this way, datasets with different numbers of spectral bands can be transformed into a fixed-dimensional representation, allowing the subsequent network modules to share a consistent architecture and improving the adaptability of the model across multiple datasets. PWC is implemented using a 1×1 convolution kernel, which performs channelwise feature projection without changing the spatial resolution. Given an input feature map $X \in \mathbb{R}^{C_{in} \times H \times W}$, the pointwise convolution transforms it into

$$Y \in \mathbb{R}^{C_{out} \times H \times W}, \quad (12)$$

where the spatial dimensions remain unchanged while the channel dimension is linearly remapped. For each spatial location (h, w) , the operation can be expressed as

$$\mathbf{y}_{h,w} = W\mathbf{x}_{h,w} + b, \quad (13)$$

where $\mathbf{x}_{h,w} \in \mathbb{R}^{C_{in}}$ and $\mathbf{y}_{h,w} \in \mathbb{R}^{C_{out}}$. This operation enables effective cross-channel interaction at each pixel location.

3.2.3 Dimensionality Reduction with 3D-CNN

To further reduce the redundancy of hyperspectral features while preserving discriminative spectral information, a 3D-CNN module is introduced for feature compression and dimensionality reduction. Unlike conventional 2D convolution, which only extracts spatial features, 3D convolution can operate along the spectral dimension and therefore model correlations among contiguous bands. In hyperspectral image classification, such a design is particularly useful because adjacent spectral bands usually exhibit strong dependencies and also contain redundancy. By exploiting this property, the 3D-CNN module can transform the input into a more compact and informative representation for subsequent processing. To adapt the 2D-CNN architecture into 3D-CNN, an additional feature dimension is incorporated, where the original

feature dimension is treated as the spectral dimension.

Given an input tensor

$$X \in \mathbb{R}^{B \times C \times D \times H \times W}, \quad (14)$$

where B , C , D , H , and W denote the batch size, channel number, spectral depth, height, and width, respectively, the proposed model first reshapes the feature representation so that 3D convolution can be applied along the spectral dimension. In this work, the 3D convolution layer uses 8 kernels with kernel size $(17, 1, 1)$ and stride $(4, 1, 1)$. This means that the convolution mainly spans 17 contiguous spectral bands while keeping the spatial dimensions unchanged. Therefore, the module focuses on spectral feature extraction and spectral dimensionality reduction rather than spatial neighborhood aggregation.

The activation value at position (d, h, w) in the m -th output feature map can be written as

$$Y_{b,m,d,h,w} = \phi \left(b_m + \sum_{i=0}^{16} W_{m,i} X_{b,1,d+i,h,w} \right), \quad (15)$$

where $W_{m,i}$ denotes the kernel weight along the spectral dimension, b_m is the bias term of the m -th filter, and $\phi(\cdot)$ denotes the nonlinear activation function. Since the kernel size along the spatial dimensions is 1×1 , the convolution is performed at each pixel location independently while aggregating local spectral responses from neighboring bands.

After the convolution operation, Batch Normalization and ReLU activation are applied to stabilize the feature distribution and improve the nonlinear representation ability of the network. The complete operation of this layer can therefore be expressed as

$$Y = \text{ReLU}(\text{BN}(\text{Conv3D}(X))) \quad (16)$$

Because the spectral stride is set to 4, the spectral dimension is compressed from D to

$$D' = \left\lfloor \frac{D - 17}{4} \right\rfloor + 1, \quad (17)$$

while the spatial dimensions remain unchanged. In this way, the proposed 3D-CNN layer effectively reduces the spectral dimensionality and removes redundant hyperspectral information, while retaining the most representative local spectral patterns. This design makes the subsequent feature learning process more efficient and helps the network focus on discriminative spectral-spatial representations.

3.2.4 Spectral Grouped Convolution (SGC) Block

To better exploit local spectral information and reduce the redundancy of hyperspectral features, a Spectral Grouped Convolution (SGC) block is introduced. SGC block is built using 3D-CNN, which are employed to capture both spatial and spectral features. Unlike conventional 2D-CNN that focus on spatial features, 3D-CNN operates over multiple contiguous bands of the HSI, allowing the model to learn complex relationships in the spectral dimension. In addition, Y. Li, et al. demonstrated that by leveraging both spectral and spatial information, 3D-CNNs can effectively extract deep spectral-spatial features from HSI data. [49] The authors

emphasize that 3D-CNN model requires fewer parameters compared to other deep learning-based methods, which makes it lighter and less prone to overfitting. Unlike conventional grouped convolution, which usually divides the input along the channel dimension, the proposed SGC performs grouping along the spectral dimension. In this way, the hyperspectral feature cube is decomposed into several local spectral groups, and each group is processed independently by a shared 3D convolutional network. It allows the model to learn discriminative spectral-spatial representations from different spectral subspaces while keeping the computation efficient.

Given an input tensor

$$X \in \mathbb{R}^{B \times C \times D \times H \times W}, \quad (18)$$

where B , C , D , H , and W denote the batch size, number of channels, spectral depth, spatial height, and spatial width, respectively, the SGC block first divides the spectral dimension into 4 local groups:

$$X = \{X^{(1)}, X^{(2)}, X^{(3)}, X^{(4)}\}, \quad (19)$$

where each spectral group is defined as

$$X^{(i)} = X[:, :, 15(i-1) : 15i, :, :], \quad i = 1, 2, 3, 4 \quad (20)$$

and

$$X^{(i)} \in \mathbb{R}^{B \times C \times 15 \times H \times W}. \quad (21)$$

Therefore, each group contains 15 consecutive spectral bands, which can be regarded as a local spectral subspace of the original hyperspectral feature cube.

For each spectral group $X^{(i)}$, three consecutive 3D convolution layers are employed to extract spectral-spatial features. The transformation process can be written as

$$F_1^{(i)} = \phi_1(\text{Conv3D}_1(X^{(i)})), \quad (22)$$

$$F_2^{(i)} = \phi_2(\text{Conv3D}_2(F_1^{(i)})), \quad (23)$$

$$F_3^{(i)} = \phi_3(\text{Conv3D}_3(F_2^{(i)})), \quad (24)$$

where Conv3D_1 , Conv3D_2 , and Conv3D_3 denote the three 3D convolution layers, and $\phi()$ represents the Batch Normalization and ReLU activation. In the proposed implementation, the three convolution kernels are set to (7,3,3), (7,3,3), (3,3,3) with output channels of 16, 32, 64, respectively.

The activation value for 3D-CNN at spatial position (x, y, z) in the j -th feature map of the i -th layer, denoted as $v_{x,y,z}^{i,j}$, is generated as follows:

$$v_{x,y,z}^{i,j} = \phi \left(b_{i,j} + \sum_{\tau=1}^{d_{l-1}} \sum_{\sigma=-\eta}^{\eta} \sum_{\rho=-\gamma}^{\gamma} \sum_{\lambda=-\delta}^{\delta} w_{\sigma,\rho,\lambda}^{i,j,\tau} \cdot v_{x+\sigma,y+\rho,z+\lambda}^{i-1,\tau} \right) \quad (25)$$

The parameters such as the bias b and the kernel weight w are trained using supervised approaches with the help of a gradient descent optimization technique.

An important property of the proposed SGC block is that the spectral dimension of each group

is progressively compressed. Since each input group has an initial spectral depth of 15, and the three convolution layers use spectral kernel sizes of 7, 7, 3 without padding, the spectral depth is reduced as

$$15 \rightarrow 9 \rightarrow 3 \rightarrow 1 \quad (26)$$

That is, the three layers gradually compress each spectral group into a highly compact representation while preserving discriminative spectral-spatial information. This process can be interpreted as a learnable local spectral aggregation mechanism.

After all spectral groups are processed independently, their outputs are concatenated along the channel dimension:

$$Y = \text{Concat} \left(F_3^{(1)}, F_3^{(2)}, F_3^{(3)}, F_3^{(4)} \right) \quad (27)$$

Each group produces 64 output channels, then the merged feature tensor can be written as

$$Y \in \mathbb{R}^{B \times 256 \times 1 \times H' \times W'} \quad (28)$$

In this way, the final representation preserves complementary information from different spectral groups, while the spectral dimension has already been reduced to 1 for each group.

Overall, the proposed SGC block plays three important roles. First, it decomposes the hyper-spectral feature cube into multiple local spectral groups, which is more consistent with the strong local continuity of adjacent spectral bands. Second, it performs spectral-spatial feature extraction within each group by shared 3D convolutional operations. Third, it significantly compresses the spectral dimension before feature fusion, which greatly reduces the computational cost of subsequent layers.

3.2.5 Squeeze-and-Excitation (SE) Block

After extracting the initial feature maps through the SGC block, the Squeeze-and-Excitation (SE) block is used to recalibrate the channel importance. The SE block helps the model focus on the most informative channels by learning a set of weights, which are used to amplify the important channels while suppressing the less relevant ones. This increases the model's capacity to capture meaningful features from the hyperspectral image. Y. Li, et al. also introduced a Spatial-Spectral Squeeze-and-Excitation module, which adaptively learns the importance of different spectral bands and neighboring pixels, which helps to suppress noise and enhance the relevant features. [50]

In the proposed model, the output feature tensor generated by the SGC block is denoted as

$$X \in \mathbb{R}^{B \times C \times D \times H \times W}, \quad (29)$$

where B is the batch size, C is the number of channels after concatenating all spectral groups, D is the spectral depth, and H and W are the spatial dimensions.

The SE block first performs a squeeze operation using global average pooling over the spectral and spatial dimensions. For the c -th channel, the squeezed descriptor is computed as

$$z_c = \frac{1}{DHW} \sum_{d=1}^D \sum_{h=1}^H \sum_{w=1}^W X_{c,d,h,w} \quad (30)$$

For the whole feature tensor, this operation generates a channel descriptor

$$\mathbf{z} \in \mathbb{R}^C, \quad (31)$$

which summarizes the global response of each channel.

After the squeeze operation, the excitation step is applied to model channel interdependencies and generate adaptive channel weights. Specifically, the descriptor $\mathbf{z}$ is passed through two fully connected layers with a bottleneck structure:

$$\mathbf{s} = \sigma(W_2 \delta(W_1 \mathbf{z})), \quad (32)$$

where $W_1 \in \mathbb{R}^{\frac{C}{r} \times C}$ and $W_2 \in \mathbb{R}^{C \times \frac{C}{r}}$ are the learnable parameters of the two fully connected layers, r denotes the reduction ratio, $\delta(\cdot)$ is the ReLU activation function, and $\sigma(\cdot)$ is the Sigmoid activation function. In the proposed model, the reduction ratio is set to

$$r = 16 \quad (33)$$

Therefore, the channel descriptor is first compressed from C dimensions to $\frac{C}{16}$ dimensions and then projected back to C dimensions, producing a set of normalized channel weights

$$\mathbf{s} = [s_1, s_2, \dots, s_C], \quad s_c \in (0, 1) \quad (34)$$

Finally, the learned channel weights are used to recalibrate the original feature tensor through channel-wise multiplication:

$$\tilde{X}_{b,c,d,h,w} = s_c \cdot X_{b,c,d,h,w} \quad (35)$$

In this way, channels that contain more discriminative spectral-spatial information are assigned larger weights, while less informative channels are suppressed. As a result, the SE block improves the representation quality of the merged features before they are forwarded to the subsequent network modules.

This design allows the network to perform adaptive channel attention on the fused multi-group spectral-spatial features, thereby improving feature selectivity and enhancing the overall classification performance.

3.3 ViT Branch

3.3.1 Dimensionality Reduction with PCA

Since hyperspectral images typically have high dimensionality, Principal Component Analysis (PCA) is applied in the ViT branch to reduce the spectral dimension while preserving the most important information. PCA helps normalize the data and reduce the risk of overfitting, making the training process more stable. This dimensionality reduction also enables the model to handle varying input sizes and provides flexibility in processing different datasets. C. Rodarmel and J. Shan also demonstrated that PCA is an effective preprocessing technique for hyperspectral image classification. [51] This study reveals that PCA can retain the most significant information and lead to a classification accuracy of about 70% using just the first few principal component bands. The authors emphasize the benefits and efficiency of PCA in handling large datasets,

which makes it a valuable tool for improving classification performance in hyperspectral image analysis.

Formally, given an original hyperspectral image

$$X \in \mathbb{R}^{H \times W \times B}, \quad (36)$$

PCA transforms it into

$$X_{\text{PCA}} \in \mathbb{R}^{H \times W \times B_p}, \quad (37)$$

where B_p denotes the number of retained principal components.

It should be noted that PCA is actually performed before patch extraction. In other words, the original hyperspectral image is first transformed into a PCA-reduced feature cube, and then local patches are extracted from both the original data and the PCA-transformed data for the two branches, respectively. In Figure 3.1, however, PCA may appear visually after the patch segmentation step for the sake of clarity and simplicity of illustration. This does not mean that PCA is executed after patch extraction in the actual implementation.

3.3.2 Global Context Capture with Pixelwise ViT

A. Dosovitskiy et al. introduced the Vision Transformer for image recognition. The study demonstrated that ViT can effectively capture both local and global dependencies within images. [52] In their framework, ViT divides an image into fixed-size patches and treats each patch as a token, then applies standard transformer layers to process these tokens to capture long-range relationships across the entire image. They also showed that the ViT model significantly improves the performance compared to CNNs with sufficient training data, especially in large-scale image recognition tasks. However, in HSI classification task, pixelwise classification is performed. Simply using the whole image does not have a significant effect because long-range dependencies between patches do not always provide meaningful information. The pixels within each patch can be very different from one another. In the proposed framework, the image is already divided into several smaller patches using a sliding window. In this case, each patch can be treated as a separate image. Within each patch, ViT is applied to perform pixelwise classification and capture the dependencies between different pixels. This is meaningful because the patch size is typically small (e.g., 13×13). In such small patches, the context of neighboring pixels is crucial because if the neighboring pixels have a certain class, the target pixel is more likely to belong to the same class. Therefore, a pixelwise ViT is introduced by treating each pixel as a token to capture global dependencies across different pixels within each patch.

Before sending the data to the final ViT part, a small transformation is performed. The spectral and spatial dimensions are combined into a single dimension, which becomes the sequence length for the ViT. The channel dimension remains as the feature dimension for the ViT, as does the batch dimension. The ViT operates on a pixelwise basis by treating each pixel as a token because the sequence length is equal to the number of pixels. It captures the global relationships between different pixels. This ViT module includes 3 transformer encoder layers and 8 attention heads, which was obtained through numerous experiments.

Given an input sequence

$$X \in \mathbb{R}^{B \times N \times C}, \quad (38)$$

where B denotes the batch size, N is the number of pixel tokens in the patch, and C is the spectral feature dimension, the input is first normalized by Layer Normalization:

$$\hat{X} = \text{LN}_1(X). \quad (39)$$

The normalized sequence is then fed into a Transformer Encoder:

$$X' = \text{TransformerEncoder}(\hat{X}), \quad (40)$$

where the encoder is composed of multi-head self-attention and feedforward layers. Through self-attention, each pixel token can interact with all other tokens in the patch, allowing the network to model long-range contextual dependencies.

After the transformer encoding, a second Layer Normalization is applied:

$$\tilde{X} = \text{LN}_2(X'). \quad (41)$$

Finally, mean pooling is performed over the token dimension to obtain the global representation of the patch:

$$z = \frac{1}{N} \sum_{i=1}^N \tilde{X}_i. \quad (42)$$

In this way, the PixelViT branch transforms the local patch into a compact feature vector that captures global contextual relationships among pixels. However, although ViT excels at modeling long-range spectral dependencies, it struggles to capture the local spatial-spectral fusion information [53]. Therefore, this refined feature embedding is then sent to the terminal feature fusion layer to be integrated with the extracted features from the CNN branch.

3.4 Feature Fusion and Classification

After feature extraction, the CNN branch and the ViT branch produce complementary representations from different perspectives. The CNN branch mainly captures local spectral-spatial structures, while the ViT branch focuses on global contextual dependencies among pixels. To combine these two types of information, the output features of both branches are first compressed into fixed-length vectors by pooling operations.

Let the pooled CNN feature and ViT feature be denoted as

$$z_{\text{cnn}} \in \mathbb{R}^{d_1}, \quad z_{\text{vit}} \in \mathbb{R}^{d_2}. \quad (43)$$

These two feature vectors are then concatenated along the feature dimension to form the fused representation:

$$z_{\text{fuse}} = \text{Concat}(z_{\text{cnn}}, z_{\text{vit}}), \quad (44)$$

where

$$z_{\text{fuse}} \in \mathbb{R}^{d_1+d_2}. \quad (45)$$

The fused feature is finally fed into a fully connected layer for classification. By concatenating the pooled CNN and ViT features, the proposed framework can jointly exploit local spectral-spatial information and global contextual information, thereby improving the discriminative

ability of the final representation. The integration of features from disparate branches has been empirically validated as a highly effective strategy for HSI classification. As demonstrated by representative models such as MFFCG [54], DFFN [55], and MFGCN [56], the fusion of multi-scale information significantly compensates for the representational gaps in single-stream architectures.

Chapter 4

Experimental Result and Analysis

The model was tested on four different benchmark datasets.

Indian Pines Dataset [57] was captured by an airborne visible/infrared imaging spectrometer (AVIRIS) sensor over Northwestern Indiana. It consists of 145×145 pixels and 200 spectral bands, with a spatial resolution of 20 meters per pixel (24 water absorption bands were excluded). This dataset consists of 10249 labeled samples and is split into 5% as the training set, 5% as the validation set, and 90% as the testing set, as detailed in Table 4.1.

Botswana Dataset [57] was captured by the Hyperion sensor aboard the NASA EO-1 satellite over the Okavango Delta, Botswana. The raw imagery possesses a spatial resolution of 30 m and spectral coverage spanning 400–2500 nm across 242 channels. Water absorption bands and low-SNR channels were excluded, resulting in a refined subset of 145 candidate features. This dataset consists of 3428 labeled samples and is split into 5% as the training set, 5% as the validation set, and 90% as the testing set, as detailed in Table 4.2.

Salinas Dataset [57] was captured by the 224-channel AVIRIS sensor over the Salinas Valley, California. Distinguished by a high spatial resolution of 3.7 meters per pixel, the captured scene spans a spatial grid of 512×217 samples. 20 water absorption and noisy channels were excluded. The resulting data, provided as at-sensor radiance, primarily documents a diverse agricultural landscape including various vegetables, fallow soils, and vineyards. This dataset consists of 54129 labeled samples and is split into 1% as the training set, 1% as the validation set, and 98% as the testing set, as detailed in Table 4.3.

WHU-Hi-LongKou Dataset [58] was captured in Longkou Town, Hubei Province, China. The resulting hyperspectral cube comprises 550×400 pixels and contains 270 spectral bands. This study area represents a typical agricultural landscape, primarily consisting of six prominent crop species: corn, cotton, sesame, broad-leaf soybean, narrow-leaf soybean, and rice. This dataset consists of 204542 labeled samples and is split into 0.2% as the training set, 0.2% as the validation set, and 99.6% as the testing set, as detailed in Table 4.4.

4.1 Experiment Setting

Because the experiment setting (training rate, validation rate and testing rate) for all the testing dataset is the same as the Hyperspectral image classification using groupwise separable convolutional vision transformer network [32], the Table 4.5, Table 4.6, Table 4.7 and Table 4.8 are their test results except the last column **DBCP**. And all the experiment was conducted on a single RTX4070 laptop GPU with 8GB memory.

Table 4.1: NUMBER OF TRAINING, VALIDATION, AND TESTING SAMPLES FOR EACH LAND COVER CLASS IN THE INDIAN PINES DATASET

Class No.	Land Cover Type	Train Num	Valid Num	Test Num
C1	Alfalfa	2	2	42
C2	Corn-notill	71	71	1286
C3	Corn-mintill	41	42	747
C4	Corn	12	12	213
C5	Grass-pasture	24	24	435
C6	Grass-trees	37	36	657
C7	Grass-pasture-mowed	1	1	26
C8	Hay-windrowed	24	24	430
C9	Oats	1	1	18
C10	Soybean-notill	49	49	874
C11	Soybean-mintill	123	123	2209
C12	Soybean-clean	30	30	533
C13	Wheat	10	10	185
C14	Woods	63	63	1139
C15	Buildings-Grass-Trees-Drives	19	19	348
C16	Stone-Steel-Towers	5	5	83
	Total	512	512	9225

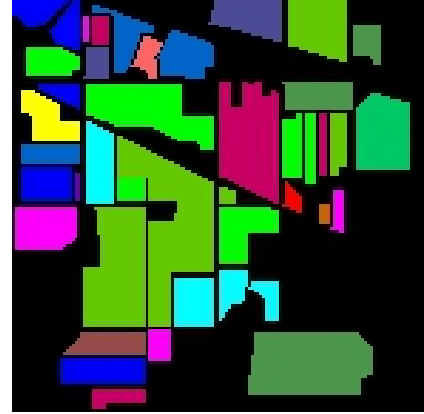


Figure 4.1: Indian Pines Ground Truth

Table 4.2: NUMBER OF TRAINING, VALIDATION, AND TESTING SAMPLES FOR EACH LAND COVER CLASS IN THE BOTSWANA DATASET

Class No.	Land Cover Type	Train Num	Valid Num	Test Num
C1	Water	14	14	242
C2	Hippo-grass	5	5	91
C3	Hippo grass-1	13	13	225
C4	Hippo grass-2	11	11	193
C5	Reeds	13	13	243
C6	Riparian	13	13	243
C7	Firescar	13	13	233
C8	Island interior	10	10	183
C9	Acacia woodlands	16	16	282
C10	Acacia shrublands	12	12	224
C11	Acacia grasslands	15	15	275
C12	Short mopanc	9	9	163
C13	Mixed mopanc	13	13	242
C14	Chalcedony	5	5	85
	Total	162	162	2924



Figure 4.2: Botswana Ground Truth

Important Note: While the same split ratios are maintained, the number of training samples/-validation samples per class may have subtle differences compared to those reported in [32]. This is because of the different rounding functions (e.g., ceiling vs. floor) used during the conversion of percentage-based rates to integer sample counts. Their training samples is slightly larger than the one used in this study. It further highlights the effectiveness of the **DBCP** model. Despite being trained on a slightly smaller number of samples, DBCP consistently achieves higher classification accuracy across all benchmark datasets.

Table 4.3: NUMBER OF TRAINING, VALIDATION, AND TESTING SAMPLES FOR EACH LAND COVER CLASS IN THE SALINAS DATASET

Class No.	Land Cover Type	Train Num	Valid Num	Test Num
C1	Brocoli_green_weeds_1	20	20	1969
C2	Brocoli_green_weeds_2	37	37	3652
C3	Fallow	20	20	1936
C4	Fallow_rough_plow	14	14	1366
C5	Fallow_smooth	27	27	2624
C6	Stubble	39	39	3881
C7	Celery	36	36	3507
C8	Grapes_untrained	113	112	11046
C9	Soil_vinyard_develop	62	62	6079
C10	Corn_senesced_green_weeds	33	33	3212
C11	Lettuce_romaine_4wk	11	11	1046
C12	Lettuce_romaine_5wk	19	19	1889
C13	Lettuce_romaine_6wk	9	9	898
C14	Lettuce_romaine_7wk	11	11	1048
C15	Vinyard_untrained	72	73	7123
C16	Vinyard_vertical_trellis	18	18	1771
	Total	541	541	53047

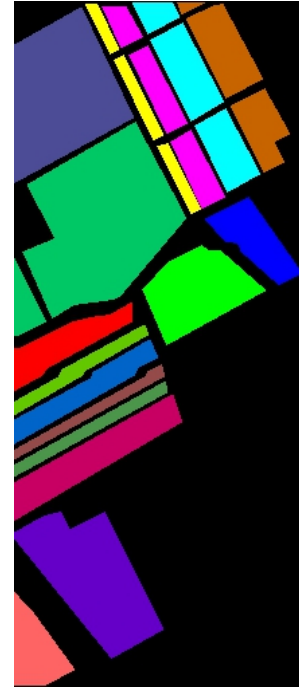


Figure 4.3: Salinas Ground Truth

Table 4.4: NUMBER OF TRAINING, VALIDATION, AND TESTING SAMPLES FOR EACH LAND COVER TYPE IN THE WHU-HI-LONGKOU DATASET

Class No.	Land Cover Type	Train Num	Valid Num	Test Num
C1	Corn	69	69	34373
C2	Cotton	17	17	8341
C3	Sesame	6	6	3019
C4	Broad-leaf soybean	126	126	62959
C5	Narrow-leaf soybean	8	8	4134
C6	Rice	24	24	11807
C7	Water	134	134	66788
C8	Roads and houses	14	14	7095
C9	Mixed weed	11	11	5208
	Total	409	409	203724

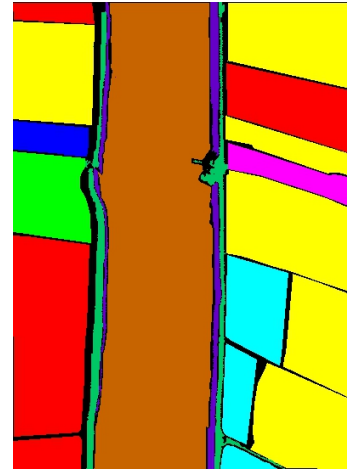


Figure 4.4: LongKou Ground Truth

Below is the hyperparameter setting for the proposed model:

- Epochs: 200
- Learning rate: 0.001 (WHU-Hi-LongKou Dataset: 0.0001)
- Weight decay: 0.001 (WHU-Hi-LongKou Dataset: 0.0001)
- Patch size: 13
- Batch size: 64
- PCA reduction: 64
- Number of runs: 10

To evaluate the robustness of the proposed model, each experiment was repeated 10 times on each dataset. The final results are reported as the mean accuracy $\pm$ standard deviation. For each run, a different random seed was used to split the dataset: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]. For each classification comparison table, the prediction maps of all 10 runs are provided below it.

Below is an overview of the alternative models in the comparison table:

- 2DCNN [59]: consists of two 2D convolutional layers followed by two fully connected layers. A max-pooling layer is applied after each convolutional layer, and the final class prediction is generated through the fully connected layers.
- SPRN [23]: detailed description in Chapter 2.1
- 3DCNN [60]: uses several 3D convolutional layers to jointly capture spectral-spatial information, followed by 1D convolutional layers and a fully connected layer for classification.
- HybridSN [20]: detailed description in Chapter 2.1
- SpectralFormer [25]: detailed description in Chapter 2.2
- SSFTT [37]: detailed description in Chapter 2.3
- GAHT [27]: detailed description in Chapter 2.2
- MorphFormer [39]: detailed description in Chapter 2.3
- CAEVT [40]: detailed description in Chapter 2.3
- GSC-ViT [32]: detailed description in Chapter 2.3

Table 4.5: Classification Results on Indian Pines Dataset

Class	2DCNN	SPRN	3DCNN	HybridSN	SpectralFormer	SSFTT	GAHT	MorphFormer	CAEVT	GSC-ViT	DBCP
1	70.24 $\pm$ 10.16	53.17 $\pm$ 17.55	58.54 $\pm$ 13.54	75.61 $\pm$ 17.86	21.95 $\pm$ 4.63	77.07 $\pm$ 22.70	29.27 $\pm$ 13.71	74.63 $\pm$ 19.37	76.10 $\pm$ 23.97	84.39$\pm$18.55	83.57 $\pm$ 13.32
2	84.03 $\pm$ 3.67	84.89 $\pm$ 5.32	79.61 $\pm$ 3.77	91.69 $\pm$ 2.72	72.62 $\pm$ 2.31	90.40 $\pm$ 2.47	93.01 $\pm$ 2.99	92.48 $\pm$ 1.63	91.69 $\pm$ 1.69	96.90$\pm$1.43	95.97 $\pm$ 2.10
3	92.32 $\pm$ 2.19	93.31 $\pm$ 3.82	76.49 $\pm$ 8.24	90.31 $\pm$ 2.41	66.88 $\pm$ 7.94	89.42 $\pm$ 1.81	94.83 $\pm$ 2.94	94.83 $\pm$ 3.48	95.72 $\pm$ 0.77	98.55$\pm$1.21	96.43 $\pm$ 2.92
4	84.69 $\pm$ 3.29	86.95 $\pm$ 2.81	51.74 $\pm$ 11.20	87.23 $\pm$ 8.60	53.99 $\pm$ 5.20	90.80 $\pm$ 6.70	91.64 $\pm$ 3.34	90.89 $\pm$ 5.21	91.36 $\pm$ 7.09	88.45 $\pm$ 7.18	92.68$\pm$7.73
5	92.87 $\pm$ 3.18	90.39 $\pm$ 4.58	88.18 $\pm$ 4.17	90.48 $\pm$ 3.59	78.53 $\pm$ 3.96	91.36 $\pm$ 3.73	90.76 $\pm$ 2.59	92.14 $\pm$ 2.44	91.36 $\pm$ 2.21	92.97 $\pm$ 2.82	95.72$\pm$1.71
6	97.47 $\pm$ 1.04	98.14 $\pm$ 0.91	98.23 $\pm$ 0.67	98.96 $\pm$ 0.53	93.88 $\pm$ 2.62	98.60 $\pm$ 0.92	99.18 $\pm$ 0.81	98.96 $\pm$ 0.72	98.93 $\pm$ 0.54	99.09 $\pm$ 0.92	99.51$\pm$0.38
7	95.20 $\pm$ 7.76	53.60 $\pm$ 25.75	77.60 $\pm$ 26.12	84.00 $\pm$ 13.39	57.60 $\pm$ 31.76	83.20 $\pm$ 19.66	69.60 $\pm$ 16.12	68.80 $\pm$ 16.08	81.60 $\pm$ 21.41	92.00 $\pm$ 14.09	95.38$\pm$6.62
8	95.26 $\pm$ 4.99	97.63 $\pm$ 3.01	98.09 $\pm$ 2.17	98.33 $\pm$ 1.63	95.63 $\pm$ 2.76	98.47 $\pm$ 1.29	99.72 $\pm$ 0.23	98.37 $\pm$ 0.76	97.81 $\pm$ 1.62	100.00$\pm$0.00	99.95 $\pm$ 0.09
9	43.33 $\pm$ 11.33	30.00 $\pm$ 10.30	41.11 $\pm$ 16.33	45.56 $\pm$ 20.61	22.22 $\pm$ 16.10	66.67 $\pm$ 12.67	17.78 $\pm$ 10.77	65.56 $\pm$ 15.48	63.33 $\pm$ 9.69	78.89 $\pm$ 8.16	87.22$\pm$18.77
10	88.75 $\pm$ 2.15	89.17 $\pm$ 2.36	79.77 $\pm$ 2.88	91.06 $\pm$ 3.48	74.33 $\pm$ 2.71	88.32 $\pm$ 5.10	93.51 $\pm$ 2.69	94.13 $\pm$ 2.34	87.79 $\pm$ 3.26	95.43 $\pm$ 2.78	95.87$\pm$1.34
11	93.15 $\pm$ 1.81	93.30 $\pm$ 0.86	89.40 $\pm$ 2.51	94.15 $\pm$ 2.68	83.91 $\pm$ 2.85	94.91 $\pm$ 1.06	95.56 $\pm$ 1.07	96.78 $\pm$ 0.51	95.34 $\pm$ 1.70	97.66 $\pm$ 0.62	98.18$\pm$0.74
12	81.95 $\pm$ 3.57	76.55 $\pm$ 12.77	71.42 $\pm$ 9.59	85.88 $\pm$ 6.62	44.19 $\pm$ 6.13	91.09 $\pm$ 6.74	91.42 $\pm$ 5.18	88.05 $\pm$ 2.82	90.86 $\pm$ 5.27	95.43$\pm$3.89	94.02 $\pm$ 1.65
13	99.03 $\pm$ 1.05	98.49 $\pm$ 1.43	99.35 $\pm$ 0.53	99.24 $\pm$ 0.81	97.84 $\pm$ 1.37	99.24 $\pm$ 0.55	99.35 $\pm$ 0.63	99.57$\pm$0.40	98.92 $\pm$ 0.76	99.57$\pm$0.40	98.59 $\pm$ 1.66
14	97.51 $\pm$ 1.40	97.05 $\pm$ 0.95	97.44 $\pm$ 2.15	97.88 $\pm$ 0.90	94.19 $\pm$ 1.83	97.82 $\pm$ 1.08	98.33 $\pm$ 1.09	99.02 $\pm$ 0.44	98.89 $\pm$ 0.82	99.21 $\pm$ 0.64	99.77$\pm$0.30
15	86.51 $\pm$ 5.40	85.65 $\pm$ 3.78	86.28 $\pm$ 5.83	88.76 $\pm$ 2.31	73.43 $\pm$ 6.14	86.86 $\pm$ 5.79	89.63 $\pm$ 5.77	90.72 $\pm$ 3.75	86.57 $\pm$ 5.13	95.45 $\pm$ 4.36	96.01$\pm$5.36
16	96.43 $\pm$ 3.28	86.67 $\pm$ 9.39	88.57 $\pm$ 3.58	95.71 $\pm$ 2.08	93.33 $\pm$ 7.21	96.19 $\pm$ 3.48	84.52 $\pm$ 9.64	90.95 $\pm$ 8.96	90.71 $\pm$ 5.75	95.48 $\pm$ 2.65	97.71$\pm$2.38
OA (%)	91.19 $\pm$ 0.82	90.84 $\pm$ 1.28	85.95 $\pm$ 2.24	93.10 $\pm$ 0.58	78.84 $\pm$ 1.21	93.15 $\pm$ 0.50	94.42 $\pm$ 0.55	94.96 $\pm$ 0.24	93.93 $\pm$ 0.02	97.12 $\pm$ 0.41	97.23$\pm$0.49
AA (%)	87.42 $\pm$ 1.07	82.18 $\pm$ 3.74	80.11 $\pm$ 3.38	88.43 $\pm$ 2.15	70.28 $\pm$ 2.90	90.03 $\pm$ 0.99	83.63 $\pm$ 1.74	89.74 $\pm$ 2.25	89.81 $\pm$ 1.58	94.34 $\pm$ 0.79	95.41$\pm$1.35
Kappa $\times$ 100	89.95 $\pm$ 0.94	89.56 $\pm$ 1.47	83.91 $\pm$ 2.59	92.12 $\pm$ 0.67	75.80 $\pm$ 1.40	92.18 $\pm$ 0.57	93.64 $\pm$ 0.63	94.25 $\pm$ 0.27	93.08 $\pm$ 0.02	96.67 $\pm$ 0.40	96.84$\pm$0.56

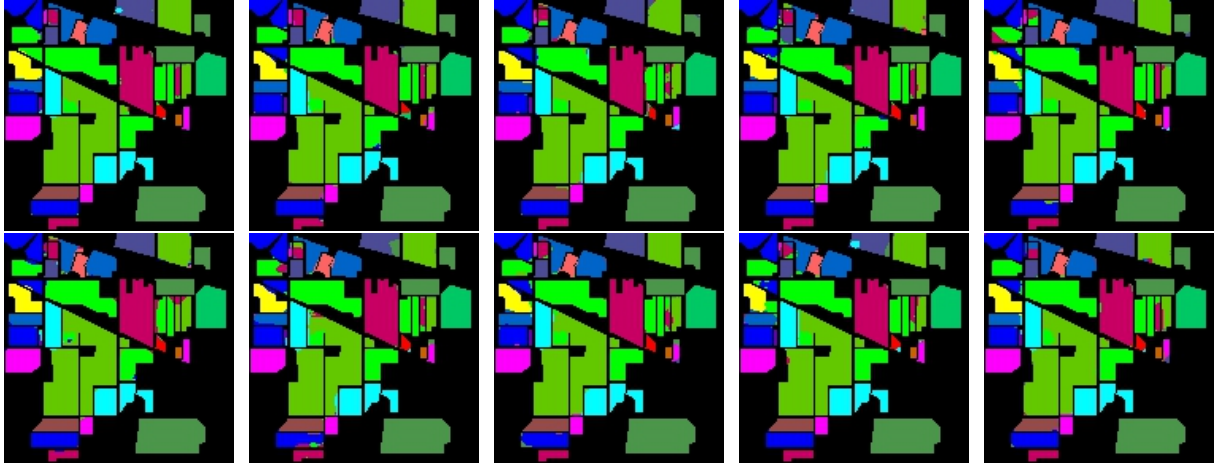


Figure 4.5: Classification maps on the Indian Pines dataset over 10 independent runs

Table 4.6: Classification Results on Botswana Dataset

Class	2DCNN	SPRN	3DCNN	HybridSN	SpectralFormer	SSFTT	GAHT	MorphFormer	CAEVT	GSC-ViT	DBCP
1	98.27±2.15	99.09±1.23	100.00±0.00	99.42±0.81	99.84±0.20	99.59±0.52	97.86±2.68	99.01±1.06	99.92±0.16	99.67±0.40	100.00±0.00
2	80.22±11.38	97.80±4.40	82.64±5.91	97.80±1.97	93.63±6.64	98.24±1.79	100.00±0.00	98.68±2.64	98.68±2.64	99.34±1.32	100.00±0.00
3	87.96±3.51	97.08±2.35	96.99±1.54	94.96±3.34	88.41±5.45	99.47±0.86	100.00±0.00	98.50±2.40	100.00±0.00	99.56±0.88	100.00±0.00
4	81.34±9.07	98.97±1.08	95.36±2.14	97.63±3.41	61.24±18.88	100.00±0.00	99.79±0.41	100.00±0.00	99.38±1.24	99.48±0.46	100.00±0.00
5	84.21±8.81	87.11±7.59	85.87±7.22	86.03±10.71	35.54±7.12	87.27±10.44	92.48±6.48	95.87±3.95	91.90±5.39	94.55±5.92	91.92±4.96
6	80.08±6.03	93.97±3.12	78.84±10.92	92.07±7.71	71.16±10.16	89.09±4.59	97.93±1.20	94.88±3.02	96.86±2.30	97.11±2.65	98.19±0.46
7	95.97±3.99	99.91±0.17	99.66±0.32	99.66±0.50	97.94±0.83	99.31±1.37	100.00±0.00	100.00±0.00	100.00±0.00	100.00±0.00	100.00±0.00
8	75.85±14.02	96.17±5.07	90.82±6.98	91.91±11.25	72.24±10.14	97.92±3.61	99.89±0.22	100.00±0.00	99.02±1.31	99.56±0.87	100.00±0.00
9	82.12±12.50	94.13±3.52	94.49±3.89	95.12±3.12	60.57±15.57	91.02±15.67	98.66±1.99	92.58±9.51	93.78±5.64	98.59±1.86	99.93±0.14
10	97.22±2.20	99.46±0.72	97.58±1.92	99.01±1.97	96.50±3.82	99.19±0.72	100.00±0.00	100.00±0.00	96.95±2.68	99.37±0.78	99.55±0.91
11	95.27±1.10	97.67±1.67	96.95±2.18	97.24±2.31	97.38±0.74	96.22±3.60	98.47±1.11	97.82±1.85	99.71±0.58	99.35±0.63	99.93±0.22
12	94.60±2.24	96.20±2.03	98.04±1.79	98.04±1.84	95.83±1.19	99.51±0.72	99.14±1.43	98.65±0.25	99.39±0.78	99.88±0.25	99.82±0.55
13	96.02±3.39	100.00±0.00	98.01±2.84	99.42±0.50	96.10±3.38	99.42±1.16	100.00±0.00	100.00±0.00	99.75±0.50	99.75±0.33	99.83±0.38
14	95.53±3.44	96.24±4.55	89.41±6.40	95.53±5.07	82.82±0.94	99.53±0.58	94.12±6.82	96.00±4.80	98.35±2.05	98.35±1.60	96.35±4.16
OA (%)	89.14±2.15	96.60±1.04	93.81±1.40	95.90±2.89	81.31±1.75	96.35±1.75	98.52±0.76	97.88±0.96	97.95±0.66	98.85±0.96	99.00±0.36
AA (%)	88.90±1.93	96.70±1.20	93.19±1.53	95.99±2.88	82.08±1.43	96.84±2.95	98.45±0.92	98.00±0.90	98.12±0.62	98.90±0.54	98.97±0.39
Kappa×100	88.23±2.33	96.32±1.13	93.29±1.51	95.55±3.13	79.76±1.88	96.05±1.89	98.39±0.82	97.70±1.04	97.78±0.72	98.75±0.52	98.92±0.39

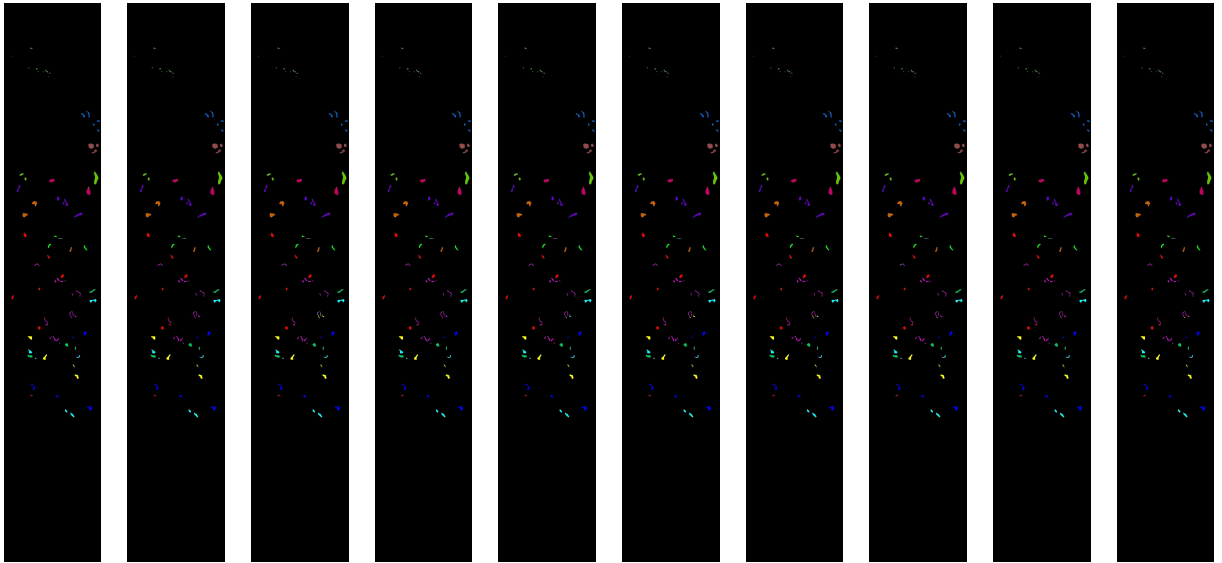


Figure 4.6: Classification maps on the Botswana dataset over 10 independent runs

Table 4.7: Classification Results on Salinas Dataset

Class	2DCNN	SPRN	3DCNN	HybridSN	SpectralFormer	SSFTT	GAHT	MorphFormer	CAEVT	GSC-ViT	DBCP
1	54.69±44.79	96.84±1.47	95.75±5.06	99.87±0.26	94.87±1.28	98.94±1.67	99.74±0.30	98.19±2.24	99.96±0.06	99.66±0.25	99.88±0.14
2	96.23±6.23	99.54±0.07	99.88±0.07	99.92±0.10	98.67±0.85	99.86±0.22	100.00±0.00	99.97±0.04	99.52±0.92	99.85±0.11	100.00±0.00
3	70.32±22.31	93.47±4.05	89.58±2.21	95.06±3.56	93.14±5.03	94.21±7.92	98.16±1.20	95.85±1.05	95.79±2.88	96.21±5.96	99.88±0.23
4	98.62±0.95	96.02±2.48	98.51±0.93	99.43±0.64	92.61±2.76	99.34±0.59	98.92±1.28	98.96±0.78	96.71±4.15	99.05±0.86	98.43±0.82
5	76.99±29.21	94.57±2.48	95.40±2.10	98.94±0.70	88.35±5.74	98.06±2.51	98.32±1.94	98.13±1.66	97.39±1.90	98.70±1.46	99.23±0.76
6	99.08±1.32	99.42±0.71	99.69±0.42	99.96±0.05	99.44±0.74	99.98±0.02	99.97±0.04	99.51±0.48	99.18±1.65	99.93±0.13	99.95±0.12
7	94.03±7.14	98.99±0.81	99.70±0.12	99.83±0.16	98.34±0.45	99.86±0.10	99.99±0.01	99.90±0.13	99.53±0.65	99.99±0.02	99.79±0.28
8	82.82±8.60	88.58±1.85	83.82±2.28	90.69±2.18	83.85±3.23	88.87±3.5	93.16±3.92	91.32±2.50	89.72±2.73	92.55±3.31	98.13±0.77
9	96.71±3.80	97.13±1.71	97.45±1.37	99.12±1.64	98.10±1.14	99.00±1.12	99.96±0.06	99.60±0.41	98.23±2.40	99.84±0.18	99.99±0.02
10	95.73±0.57	92.43±1.05	92.80±1.46	95.62±1.79	87.68±6.32	96.18±2.80	97.02±2.03	95.31±3.46	94.89±2.97	97.38±2.10	98.90±1.61
11	64.74±31.13	93.12±3.93	73.26±8.11	97.48±2.00	76.16±4.56	95.36±5.15	98.72±0.87	96.87±3.49	94.54±5.44	96.96±2.26	99.98±0.04
12	94.22±4.36	99.36±0.51	98.59±1.19	99.84±0.19	95.71±1.29	99.72±0.15	99.76±0.24	99.86±0.16	99.11±1.23	98.54±1.14	99.28±1.11
13	99.47±0.54	99.76±0.30	99.00±1.68	97.02±4.76	94.37±6.84	98.26±2.72	98.55±1.78	99.00±1.45	98.49±1.70	99.82±0.26	98.76±1.41
14	92.32±7.09	97.67±1.67	97.98±0.64	98.28±0.83	96.93±1.60	95.77±3.14	99.24±0.21	98.59±0.72	98.93±1.05	98.86±0.98	99.15±0.75
15	76.72±15.80	85.94±3.88	74.31±5.49	82.98±5.02	76.46±6.63	85.66±6.97	90.93±3.99	92.19±2.45	86.87±4.87	92.74±3.02	97.98±1.50
16	78.22±26.28	88.74±2.25	91.90±1.52	95.98±1.93	91.53±4.29	97.30±0.98	97.01±2.83	96.79±1.74	96.47±2.10	97.78±1.70	98.53±2.85
OA (%)	86.21±5.05	93.49±0.64	90.69±0.75	94.86±0.60	90.00±1.36	94.72±0.50	96.81±0.34	96.21±0.45	94.79±0.37	97.15±0.47	99.06±0.41
AA (%)	85.68±7.11	95.10±0.78	92.98±0.49	96.88±0.57	91.64±1.20	96.65±0.62	98.09±0.21	97.50±0.22	96.58±0.56	97.99±0.62	99.24±0.33
Kappa×100	84.63±5.67	92.76±0.71	89.64±0.83	94.28±0.67	88.87±1.51	94.13±0.56	96.45±0.38	95.79±0.50	94.20±0.41	96.47±0.83	98.95±0.46

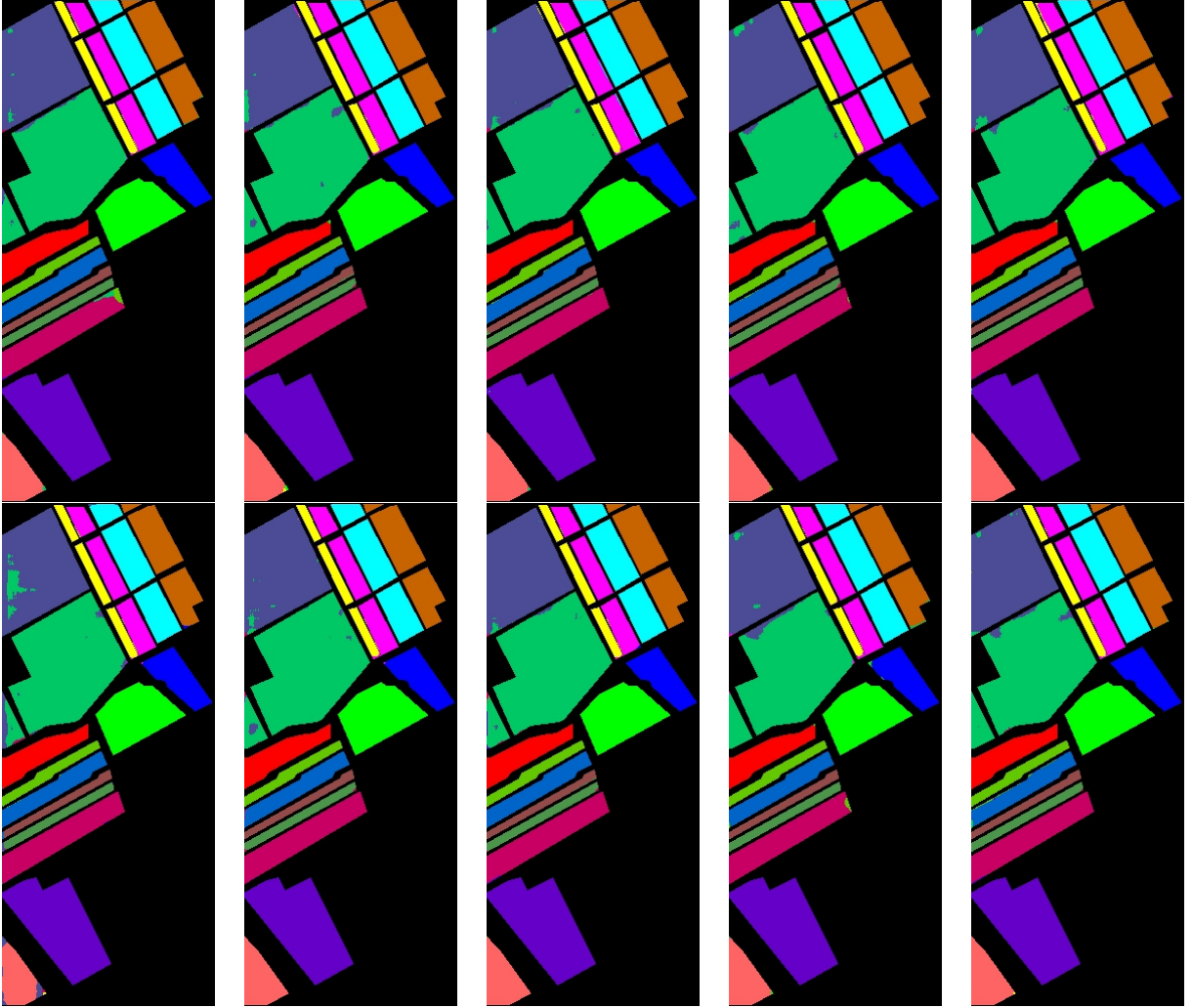


Figure 4.7: Classification maps on the Salinas dataset over 10 independent runs

Table 4.8: Classification Results on WHU-Hi-LongKou Dataset

Class	2DCNN	SPRN	3DCNN	HybridSN	SpectralFormer	SSFTT	GAHT	MorphFormer	CAEVT	GSC-ViT	DBCP
1	96.70±4.00	97.84±1.52	98.03±0.72	99.55±0.31	99.01±0.52	99.50±0.34	99.68±0.16	99.46±0.51	99.50±0.27	99.78±0.14	99.93±0.05
2	50.72±42.09	85.20±4.75	77.46±3.84	95.15±2.41	81.62±9.33	96.09±2.18	96.87±1.08	97.16±1.42	94.13±4.09	97.90±0.91	96.53±2.47
3	0.00±0.00	47.19±33.08	54.78±13.54	78.90±7.32	92.89±2.72	85.27±7.26	86.88±7.95	89.20±3.61	88.26±6.22	90.50±4.93	94.67±3.96
4	95.84±1.81	97.85±0.61	97.37±0.60	97.59±0.71	96.99±1.05	99.08±0.38	99.13±0.47	99.24±0.37	98.44±0.53	98.94±0.51	99.62±0.16
5	15.78±31.55	20.68±6.82	41.49±13.28	79.64±2.59	66.47±9.33	83.72±3.70	85.22±4.50	82.01±4.38	82.16±4.89	85.88±5.60	94.88±4.41
6	66.69±22.52	91.84±7.15	93.97±4.65	96.26±2.07	88.08±2.13	96.76±2.56	98.73±0.68	97.71±1.64	98.24±1.62	97.14±2.42	98.79±2.58
7	99.98±0.02	99.95±0.07	99.98±0.02	99.92±0.14	99.90±0.16	99.90±0.14	99.84±0.19	99.93±0.09	99.82±0.25	99.89±0.18	99.99±0.01
8	78.74±27.83	85.40±6.39	87.79±1.36	88.99±2.01	75.09±5.85	93.14±2.45	92.55±2.55	92.60±1.73	91.81±4.09	96.05±1.88	94.41±2.68
9	39.36±19.96	56.89±10.97	68.61±10.47	80.16±5.38	44.60±7.80	83.53±6.87	81.04±7.60	76.72±7.48	87.30±7.17	83.73±9.40	76.46±11.03
OA (%)	88.72±3.91	93.87±1.21	94.49±0.51	97.12±0.26	94.36±0.61	98.04±0.23	98.18±0.31	98.02±0.33	97.89±0.17	98.65±0.20	98.68±0.31
AA (%)	60.42±8.51	75.87±5.31	79.94±2.07	90.68±0.83	82.74±1.89	93.00±1.21	93.33±1.94	92.67±1.31	93.30±1.05	94.42±1.71	95.03±1.24
Kappa×100	84.96±5.29	91.89±1.63	92.72±0.68	96.21±0.34	92.57±0.80	97.42±0.30	97.61±0.41	97.40±0.44	97.22±0.23	97.85±0.52	98.26±0.41

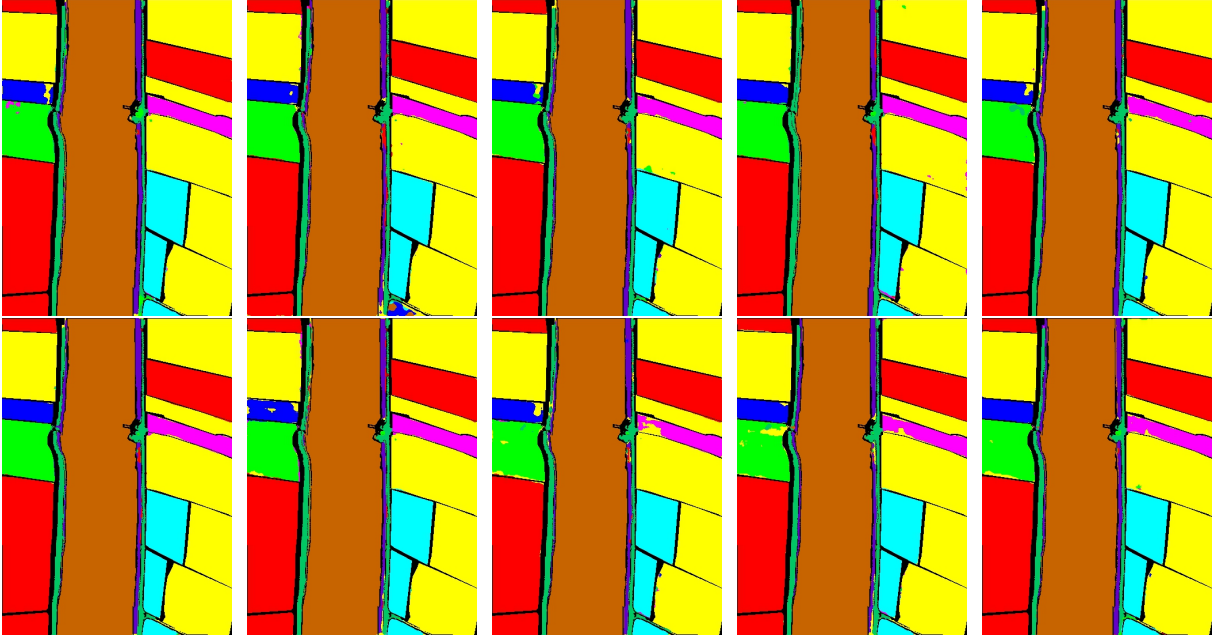


Figure 4.8: Classification maps on the WHU-Hi-Longkou dataset over 10 independent runs

Table 4.9: Ablation Study of Different Components on Indian Pines Dataset

Class	Only CNN	Only pixelViT	noSGCBlock	noSEBlock	DBCP (Complete)
1	80.48±16.35	61.90±14.68	82.14±9.48	88.33±11.73	83.57±13.32
2	96.17±1.66	94.39±2.20	96.15±1.64	96.78±2.03	95.97±2.10
3	94.31±6.73	94.54±2.99	95.41±2.82	96.31±2.85	96.43±2.92
4	85.45±9.95	90.89±4.50	92.16±7.23	87.98±6.77	92.68±7.73
5	93.31±1.39	93.75±3.07	95.29±1.77	96.11±1.62	95.72±1.71
6	98.74±1.37	97.66±1.30	98.30±1.32	99.47±0.58	99.51±0.38
7	88.08±16.27	67.69±25.23	83.85±17.10	86.15±12.89	95.38±6.62
8	99.98±0.07	99.60±0.69	99.79±0.42	99.95±0.09	99.95±0.09
9	80.00±25.60	45.56±23.01	66.67±19.56	80.00±15.95	87.22±18.77
10	95.41±1.84	94.49±2.00	95.29±2.63	94.31±3.09	95.87±1.34
11	98.87±0.85	97.28±0.93	97.57±0.35	98.27±0.89	98.18±0.74
12	94.84±4.13	91.69±2.12	94.37±2.98	94.07±2.49	94.02±1.65
13	97.24±2.17	94.32±4.40	95.30±4.58	99.30±0.91	98.59±1.66
14	99.80±0.29	99.39±0.59	99.63±0.34	99.85±0.16	99.77±0.30
15	90.72±9.37	95.09±10.77	96.55±6.77	93.76±6.50	96.01±5.36
16	96.51±3.02	91.45±4.55	90.84±7.45	99.16±1.21	97.71±2.38
OA (%)	96.64±0.91	95.61±0.83	96.66±0.40	97.05±0.58	97.23±0.49
AA (%)	93.12±2.58	88.10±2.71	92.46±2.08	94.36±1.72	95.41±1.35
Kappa×100	96.16±1.05	94.99±0.95	96.19±0.45	96.63±0.66	96.84±0.56

Table 4.10: Model Parameters Comparison Across Four HSI Datasets

Dataset	Model	Parameters/K
Indian Pines	2DCNN	1718.6
	SPRN	183.35
	3DCNN	161.68
	HybridSN	512.22
	SpectralFormer	345.53
	SSFTT	931.85
	GAHT	830.8
	MorphFormer	199.6
	CAEVT	359.95
	GSC-ViT	103.18
	DBCP	312.66
Salinas	2DCNN	1718.16
	SPRN	183.35
	3DCNN	263.6
	HybridSN	512.22
	SpectralFormer	355.29
	SSFTT	950.28
	GAHT	972.62
	MorphFormer	202.8
	CAEVT	359.95
	GSC-ViT	104.21
	DBCP	313.69
Botswana	2DCNN	1581.97
	SPRN	183.35
	3DCNN	179.59
	HybridSN	512.22
	SpectralFormer	230.72
	SSFTT	678.34
	GAHT	800.72
	MorphFormer	194.93
	CAEVT	262.67
	GSC-ViT	103.05
	DBCP	297.82
WHU-Hi-LongKou	2DCNN	1869.32
	SPRN	183.35
	3DCNN	201.99
	HybridSN	512.22
	SpectralFormer	543.52
	SSFTT	1253.95
	GAHT	1514.2
	MorphFormer	255.14
	CAEVT	454.92
	GSC-ViT	126.31
	DBCP	328.47

4.2 Performance Comparison

Table 4.5 to Table 4.8 report the classification performance of all compared methods on four benchmark hyperspectral datasets. Overall, **DBCP** achieves the best performance on all four datasets in terms of OA, AA, and Kappa, demonstrating its strong effectiveness and generalization ability across scenes with different spatial structures and class distributions. In addition to the improvements in overall metrics, DBCP also achieves the best classwise accuracy in a large proportion of categories, especially on several hard or easily confused classes, which further verifies the advantage of the proposed architecture.

4.2.1 Evaluation on Indian Pines Dataset

The classification results on the Indian Pines dataset are presented in Table 4.5. Among all compared methods, DBCP achieves the best overall performance, with an OA of $97.23\% \pm 0.49\%$, an AA of $95.41\% \pm 1.35\%$, and a Kappa value of $96.84\% \pm 0.56\%$. Compared with the strongest competing method GSC-ViT, DBCP improves OA from 97.12% to 97.23%, AA from 94.34% to 95.41%, and Kappa from 96.67% to 96.84%. Although the OA gain is relatively small, the improvement in AA is more obvious, indicating that DBCP not only improves overall pixel classification accuracy but also has a more balanced recognition ability across different classes.

From the classwise results, DBCP ranks first in 10 out of 16 classes, including Classes 4, 5, 6, 7, 9, 10, 11, 14, 15, and 16. Its advantage is especially clear on several difficult categories. For example, on Class 9, DBCP reaches 87.22%, which is substantially higher than GSC-ViT (78.89%), SSFTT (66.67%), and HybridSN (45.56%). On Class 7, DBCP achieves 95.38%, outperforming GSC-ViT (92.00%) and clearly exceeding several other methods that show relatively unstable results on this class.

It is worth noting that DBCP is not the best model on every class. GSC-ViT performs better on Classes 1, 2, 3, 8, and 12, while MorphFormer and GSC-ViT achieve the best result on Class 13. However, DBCP still delivers the highest OA, AA, and Kappa on Indian Pines, indicating that its overall classification behavior is more robust and more balanced than the competing methods.

4.2.2 Evaluation on Botswana Dataset

The classification results on the Botswana dataset are presented in Table 4.6. DBCP again achieves the best overall performance, with an OA of $99.00\% \pm 0.36\%$, an AA of $98.97\% \pm 0.39\%$, and a Kappa value of $98.92\% \pm 0.39\%$. Compared with the strongest competing method GSC-ViT, DBCP improves OA from 98.85% to 99.00%, AA from 98.90% to 98.97%, and Kappa from 98.75% to 98.92%. Since most advanced methods already achieve very high accuracies on this dataset, further improvement is relatively difficult. Therefore, the consistent gains obtained by DBCP in all three overall metrics are still meaningful and demonstrate its superiority even in a near saturated classification setting.

From the classwise results, DBCP achieves the best result in 9 out of 14 classes. It reaches 100% accuracy on Classes 1, 2, 3, 4, 7, and 8, and also obtains the best performance on Classes 6, 9, and 11. The gain is most obvious on Class 9, where DBCP achieves 99.93%, surpassing GSC-ViT (98.59%), GAHT (98.66%), and HybridSN (95.12%). On Class 6, DBCP reaches 98.19%, which is higher than GSC-ViT (97.11%), CAEVT (96.86%), and MorphFormer (94.88%).

These results indicate that DBCP remains highly effective not only on easy categories but also on classes where better spectral-spatial discrimination is required.

It is worth noting that DBCP is not the best model on every class. MorphFormer performs best on Class 5, SSFTT and MorphFormer achieve the best result on Class 10, GSC-ViT performs best on Class 12, several methods obtain 100% on Class 13, and SSFTT performs best on Class 14. Even so, DBCP still provides the strongest aggregate performance on Botswana, showing that its improvements are consistent at the dataset level rather than being limited to only a few classes.

4.2.3 Evaluation on Salinas Dataset

The classification results on the Salinas dataset are presented in Table 4.7. DBCP achieves the best overall performance among all methods, with an OA of $99.06\% \pm 0.41\%$, an AA of $99.24\% \pm 0.33\%$, and a Kappa value of $98.95\% \pm 0.46\%$. Compared with the strongest competing method GSC-ViT, DBCP improves OA by 1.91%, AA by 1.25%, and Kappa by 2.48%. This is the largest overall margin among the four datasets, which suggests that the proposed model is particularly effective on Salinas and can exploit its spectral-spatial characteristics more thoroughly than the competing methods.

From the classwise results, DBCP ranks first or shares first place in 9 out of 16 classes. It achieves 100% on Class 2 and obtains the best results on Classes 3, 5, 8, 9, 10, 11, 15, and 16. The improvement is especially remarkable on several challenging classes. For instance, on Class 8, DBCP reaches 98.13%, which is much higher than GSC-ViT (92.55%), GAHT (93.16%), and HybridSN (90.69%). On Class 11, DBCP obtains 99.98%, clearly outperforming GAHT (98.72%), HybridSN (97.48%), and SSFTT (95.36%). On Class 15, DBCP achieves 97.98%, surpassing GSC-ViT (92.74%), MorphFormer (92.19%), and GAHT (90.93%). These large class-wise gains indicate that DBCP is particularly strong at recognizing categories that are more difficult to separate using conventional spectral-spatial modeling strategies.

It is worth noting that DBCP is not the best model on every class. For example, CAEVT performs best on Class 1, HybridSN on Class 4, SSFTT on Class 6, GSC-ViT on Class 7, MorphFormer on Class 12, GAHT on Class 14, and GSC-ViT on Class 13. However, DBCP still achieves the highest OA, AA, and Kappa on Salinas by a clear margin, showing that it provides the most competitive and most stable overall classification performance on this dataset.

4.2.4 Evaluation on WHU-Hi-LongKou Dataset

The classification results on the WHU-Hi-LongKou dataset are presented in Table 4.8. DBCP once again achieves the best overall performance, with an OA of $98.68\% \pm 0.31\%$, an AA of $95.03\% \pm 1.24\%$, and a Kappa value of 98.26 ± 0.41 . Compared with the second-best OA result from GAHT, DBCP improves OA from 98.18% to 98.68%. Compared with GSC-ViT, DBCP improves AA from 94.42% to 95.03% and Kappa from 97.85 to 98.26. These results confirm that DBCP maintains strong performance even on a more challenging real-scene dataset with complex class distributions and strong inter-class similarity.

From the classwise results, DBCP ranks first in 6 out of 9 classes, namely Classes 1, 3, 4, 5, 6, and 7. Its advantage is particularly significant on Class 5, where DBCP achieves

94.88%, outperforming GSC-ViT (85.88%), GAHT (85.22%), SSFTT (83.72%), and HybridSN (79.64%). DBCP also performs best on Class 3 with 94.67%, which is higher than GSC-ViT (90.50%) and SpectralFormer (92.89%). On Classes 1, 4, 6, and 7, DBCP also achieves the highest accuracies, showing that the proposed model can effectively handle both dominant and difficult land-cover categories in the LongKou scene.

It is worth noting that DBCP is not the best model on every class. GSC-ViT performs better on Classes 2 and 8, while CAEVT achieves the best result on Class 9 with 87.30%, compared with 76.46% for DBCP. This suggests that there is still room for improvement in handling certain highly specific or confusing categories. Even so, DBCP still achieves the best overall OA, AA, and Kappa on the LongKou dataset, which indicates that its overall advantages remain significant at the dataset level.

4.3 Ablation Study

To rigorously evaluate the individual contribution and the structural necessity of each functional module within the DBCP framework, an ablation study was conducted on the Indian Pines dataset and the results are presented in Table 4.9. The experimental design deconstructs the architecture into five configurations: a standalone CNN branch, a standalone pixelwise ViT branch, and two degenerate versions of the proposed model, those excluding the Spatial-Grouping Convolution (SGC) block, the Squeeze-and-Excitation (SE) block, and the full DBCP model. Among all variants, the complete DBCP model achieves the best overall performance, with an OA of $97.23\% \pm 0.49\%$, an AA of $95.41\% \pm 1.35\%$, and a Kappa coefficient of $96.84\% \pm 0.56\%$.

Compared with the two single-branch variants, Only CNN and Only pixelViT, the complete model consistently obtains higher OA, AA, and Kappa values. In particular, Only pixelViT achieves the lowest performance among all settings, with an OA of $95.61\% \pm 0.83\%$ and an AA of $88.10\% \pm 2.71\%$, which suggests that relying only on the transformer branch is insufficient for fully capturing discriminative spectral-spatial information. Although Only CNN performs better than Only pixelViT, its OA ($96.64\% \pm 0.91\%$) and AA ($93.12\% \pm 2.58\%$) are still lower than those of the complete model, indicating that local convolutional features alone are also not enough. These results demonstrate that the combination of CNN-based local feature extraction and pixelViT-based global modeling is more effective than either branch used independently.

The noSGCBlock configuration represents a structural simplification of the CNN branch. Specifically, it means the model removes the multi-branch architecture, which typically consists of four parallel 3D-CNN spectral sub-branches—and replaces them with a single 3D-CNN branch. While the full DBCP model uses these parallel branches to capture fine-grained features across different spectral segments, the no SGC version processes the entire data cube through a single path. This leads to a loss in discriminative power, as evidenced by a drop in OA from 97.23% to 96.55% and a significant drop in AA from 95.41% to 92.46%. It proves that the multi-branch grouping is essential for handling the high dimensionality and spatial variability of hyperspectral image data.

Furthermore, the exclusion of the SE block leads to a drop in AA from 95.41% and to 94.36%. It demonstrates that the SE block enhances the model’s adaptability across diverse and imbalanced classes. By implementing a channel-wise recalibration mechanism, the SE block adaptively weights the importance of different spectral channels. This allows the model to em-

phasize informative bands while suppressing redundant noise, which is crucial for distinguishing materials with high inter-class similarity. Consequently, the SE block ensures a more precise fit for a wider variety of land cover categories, particularly for classes where training samples are limited.

Overall, the ablation study confirms that each component of DBCP contributes positively to the final performance, and the complete architecture provides the best balance between local feature extraction, global dependency modeling, and channel-wise feature recalibration.

4.4 Computational Efficiency Analysis

The computational efficiency of the proposed DBCP model is evaluated through a comparative analysis of network parameters across four benchmark datasets. The results are presented in Table 4.10. This table is also cited from [32] except the last row **DBCP** for each datasets.

A significant reduction in model complexity is achieved by the DBCP architecture compared to traditional deep learning frameworks. On the Indian Pines dataset, the parameter count of the DBCP model is measured at 312.66 K. Although this is higher than the lightweight GSC-ViT (103.18 K), it represents a great improvement over standard convolutional neural networks. Specifically, the number of parameters in DBCP is found to be approximately 81.8% lower than that of the 2DCNN (1718.6 K) and 39.0% lower than the HybridSN (512.22 K). On the WHU-Hi-LongKou dataset, the parameter growth is controlled at 328.47 K, which is significantly more efficient than SSFTT (1253.95 K) and GAHT (1514.2 K). These results indicate that a balance between model depth and computational cost is maintained by the dual-branch structure, making it suitable for deployment in resource-constrained environments.

It is also worth noting that the parameter count of the DBCP model remains stable at approximately 300 K across all four diverse datasets. The consistency is an advantage, particularly when compared to models like SSFTT or GAHT, whose complexity is observed to fluctuate or increase drastically depending on the input dimensions. In HSI classification, a model size within the 300 K range can be classified as a relatively low-parameter standard, especially for hybrid architectures. Given the dual-branch nature of the DBCP architecture, which is designed to simultaneously extract spectral-spatial features and global spectral context, the moderate increase in parameter count relative to single-branch models is considered acceptable. This slight trade-off in model size is justified by the enhanced feature representation capabilities and superior performance.

4.5 Brief Summary

In summary, the experimental results on all four datasets consistently demonstrate the superiority of DBCP. Compared with representative CNN-based, Transformer-based methods, and hybrid CNN-Transformer, DBCP achieves better overall accuracy and better classwise performance on difficult classes. These results verify that the proposed design can effectively enhance spectral-spatial feature extraction and improve hyperspectral image classification performance across multiple benchmark datasets.

Chapter 5

Reflection

5.1 Project Management

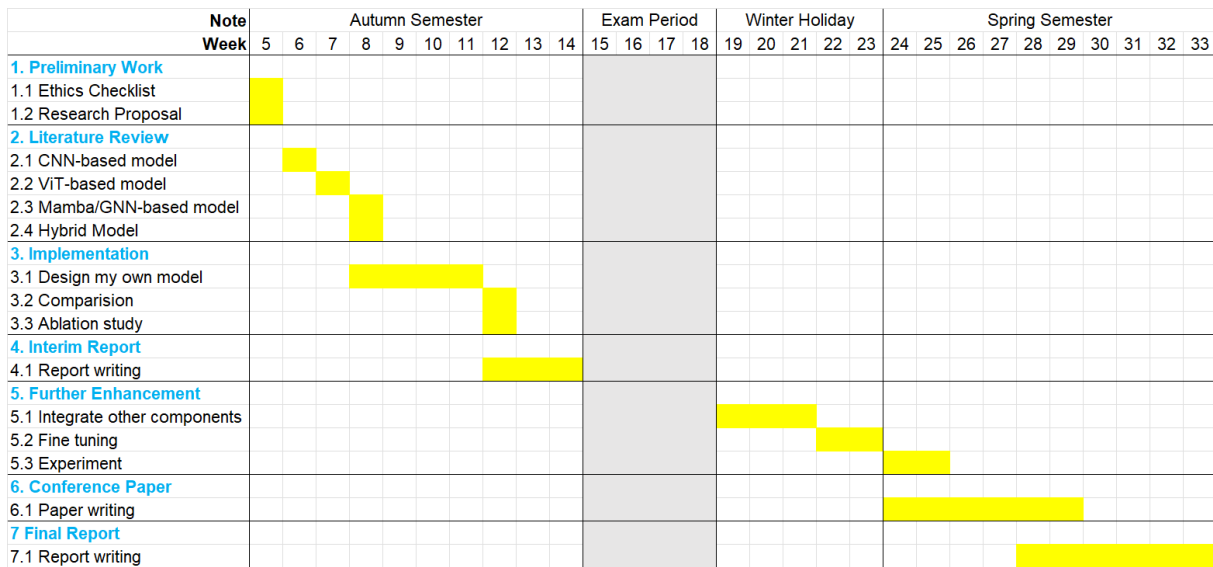


Figure 5.1: Original Plan

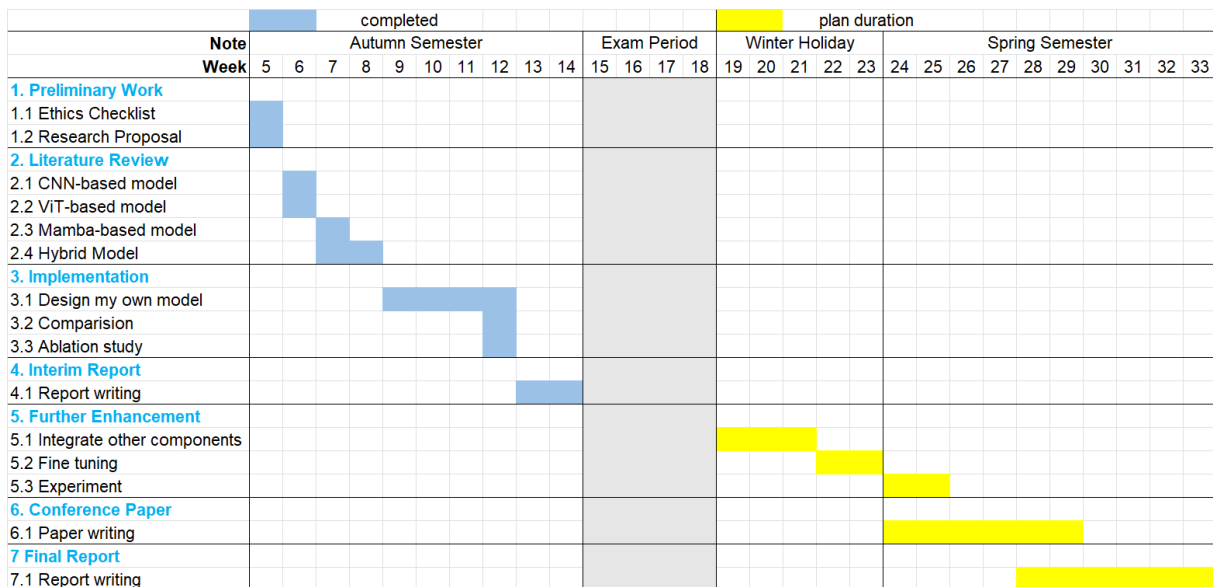


Figure 5.2: Interim Report Plan

		completed																																
	Note	Autumn Semester													Exam Period				Winter Holiday					Spring Semester										
	Week	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33				
1. Preliminary Work																																		
1.1 Ethics Checklist																																		
1.2 Research Proposal																																		
2. Literature Review																																		
2.1 CNN-based model																																		
2.2 ViT-based model																																		
2.3 Mamba-based model																																		
2.4 Hybrid Model																																		
3. Implementation																																		
3.1 Design my own model																																		
3.2 Comparison																																		
3.3 Ablation study																																		
4. Interim Report																																		
4.1 Report writing																																		
5. Further Enhancement																																		
5.1 Integrate other components																																		
5.2 Fine tuning																																		
5.3 Experiment																																		
6 Final Report																																		
6.1 Report writing																																		

Figure 5.3: Final Timetable

The project management includes three phases: the Original Plan (Figure 5.1), the Interim Report Plan (Figure 5.2), and the Final Plan (Figure 5.3). Based on the comparison of these three plans, some adjustments have been made:

- **Preliminary Work:** As planned, the ethics checklist and research proposal were completed in weeks 5 as shown in both the original and updated plans.
- **Literature Review:** The study of CNN-based models, ViT-based models, Mamba-based models, and hybrid models was largely completed according to the original timeline, but the updated plan reflects a slight shift in the schedule. Additional time was devoted to the study of hybrid models.
- **Implementation:** The actual implementation stage started one week later while it still lasted for five weeks as planned. The updated timeline reflects the adjustments.
- **Interim Report:** The writing of the interim report was planned for weeks 12-14 of the Autumn semester. However, due to the one week delay of the implementation stage, the interim report was completed in weeks 13-14.
- **Further Enhancement:** Although the original plan designated the winter holiday for further enhancement, the actual execution of this phase was rescheduled to start at Week 24 due to personal commitments. In the first month of the spring semester, the transition from the preliminary **S3ViT** (Spectral–Spatial Squeeze Vision Transformer) model to the current **DBCP** (Dual Branch Convolution with Pixelwise Vision Transformer) model was finalized. Specifically, the architecture was upgraded from a serial CNN-ViT connection to a more sophisticated dual-branch structure with SGC (Separate Groupwise Convolution) block. This fundamental change allows for more effective extraction of local spatial features and global spectral context, which greatly enhances the model’s classification performance. Additionally, the fine-tuning process and experimental evaluations were successfully completed within the subsequent two weeks.
- **Conference Paper:** Due to the rescheduled timeline of the Further Enhancement phase, the plan to prepare a conference paper has been cancelled.

- **Final Report:** The final report writing was completed during week 28–33 in the Spring semester as scheduled. Despite the earlier adjustments to the plan, this terminal phase remained on track.

5.2 Risk Management

Risk management was carried out throughout the project to reduce the likelihood that technical, scheduling and resource-related issues would negatively affect the progress of the research. Several major risks were identified and corresponding mitigation strategies were considered:

- **Limited Computational Resources and Long Training Time:** HSI classification experiments are computationally demanding, especially when multiple benchmark datasets and repeated runs are involved. One run may take an hour or even longer, making the hyperparameter tuning even more time consuming. This risk was addressed by controlling the model complexity through appropriate patch size selection, spectral dimensionality reduction and efficient architectural design.
- **Experimental Reproducibility Issue:** At the early stage of the project, experimental reproducibility was a significant challenge. Although random seeds had already been set, repeated runs still produced inconsistent results, which affected the reliability of model evaluation. To address this issue, extensive troubleshooting was carried out through literature search and detailed comparison with reference implementations. After identifying and correcting the sources of inconsistency in the experimental pipeline, the results became reproducible. This helped avoid unnecessary repeated training and saved time.
- **Unsatisfactory Model Performance:** There was a possibility that the proposed model might not outperform existing methods and some newly introduced modules would not contribute effectively to the final results. To reduce this risk, continuous literature review and iterative model refinement were carried out during the project. Ablation studies were also performed to evaluate the contribution of each component and to guide the transition from the interim S3ViT model to the final DBCP model.
- **Time Management:** Some stages of the project took longer than initially expected, and there were also noticeable delays during certain periods, particularly the gap over the winter holiday. This risk was mitigated by adjusting the timetable and prioritizing the completion of the final model, core experiments, and dissertation writing over secondary objectives such as conference paper. This helped ensure that all essential tasks were completed before the deadline.

5.3 Model Evolution

During the Further Enhancement stage, the model was upgraded from S3ViT to DBCP. To provide a clearer perspective on this architectural evolution, below is the retained components from S3ViT and novel enhancements in DBCP:

- **Retained Components from S3ViT:** The PCA pre-processing stage and the Pixelwise Vision Transformer core were retained from the S3ViT framework. These elements

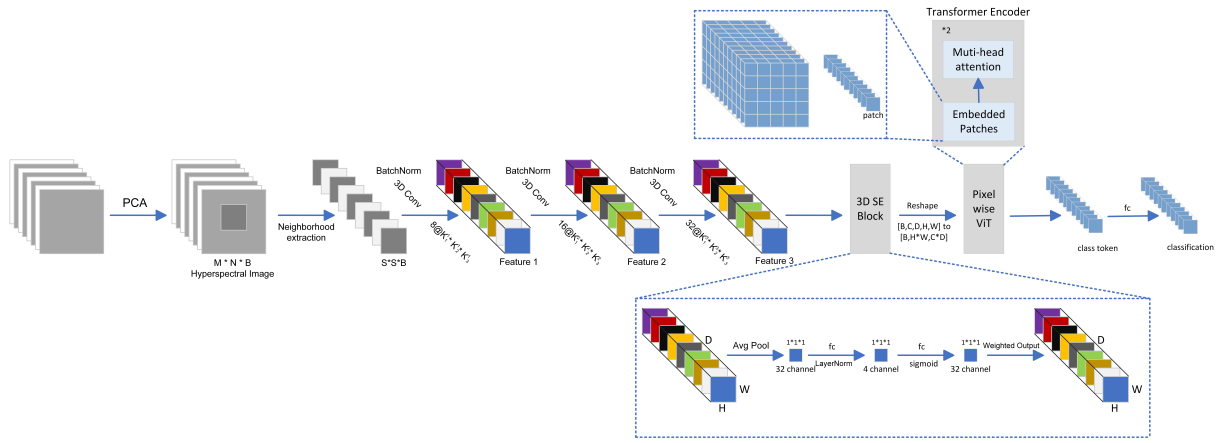


Figure 5.4: S3ViT (Interim Report Model)

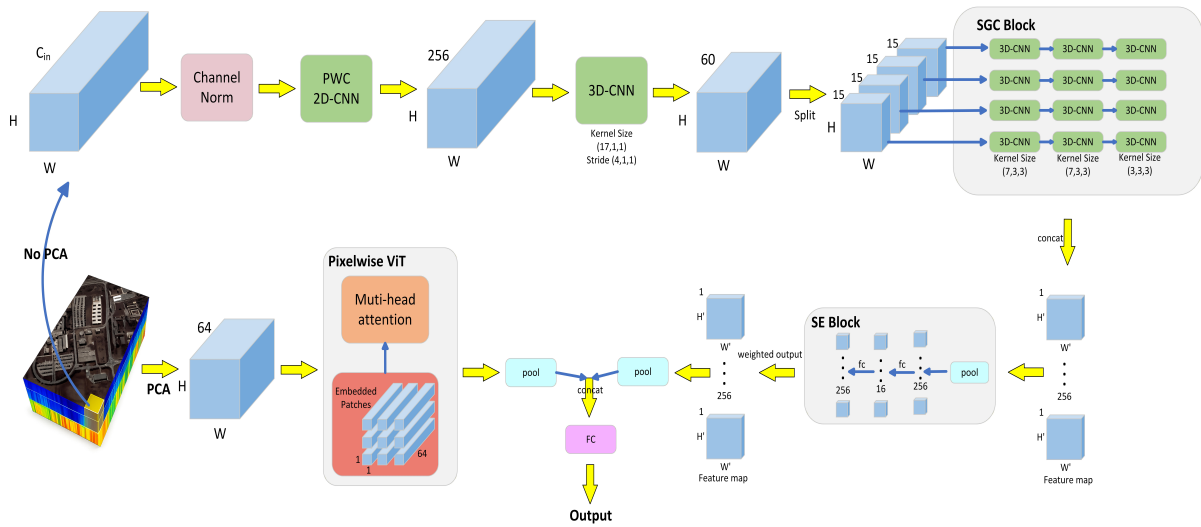


Figure 5.5: DBCP (Final Model)

continue to provide the foundation for spectral dimension reduction and long-range spatial dependency modeling. Additionally, the use of 3D-CNN as a basic building block for 3D data processing remains a key commonality, which ensures the model's ability to extract the spectral-spatial information of hyperspectral cubes. Furthermore, the Squeeze-and-Excitation (SE) Block is also retained, which continues to serve as a critical channelwise attention mechanism for recalibrating feature maps.

- **Novel Enhancements in DBCP:** The most fundamental change is the transition from a serial CNN-ViT connection to a Dual-Branch structure. In this new paradigm, the CNN-based path and the pixelwise ViT path operate in parallel rather than sequentially. This allows the model to preserve raw spectral-spatial relationships in one branch while simultaneously extracting high-level semantic features in the other, preventing the feature dilution often found in deep serial pipelines. DBCP also introduced the Separate Groupwise Convolution (SGC) Block, which replaces the single-stream convolution layer with multiple branches in the spectral dimension and extract spectral-spatial information separately and then concatenate them. Additionally, a Channel Normalization layer and

a Pointwise Convolution (PWC) 2D-CNN layer were integrated to refine spectral features before they enter the 3D-CNN stage. Furthermore, DBCP replaced the static PCA stage with a trainable 3D-CNN module within the CNN branch. While S3ViT relies on PCA to linearly project high-dimensional spectral data into a reduced space before feature extraction, DBCP leverages 3D-CNN kernels to perform dimensionality reduction. It allows the model to preserve critical spectral-spatial signatures that might be discarded by variance-based reduction methods.

By maintaining those useful modules in S3ViT while integrating dual-branch architecture, SGC block, Channel Normalization, PWC layer and 3D-CNN dimensionality reduction, DBCP achieves a superior classification accuracy.

5.4 Reflection on original plan

The original project plan provided a useful overall structure for managing the research process and it helped maintain the direction of the project. Several parts of the project were completed broadly in line with the original schedule such as the preliminary work, the literature review, and the final report writing stage.

However, the original plan was not exactly followed. Some stages of the project took longer than initially expected, and there were also noticeable delays during certain periods, particularly the gap over the winter holiday, which led to the cancellation of the conference paper plan. Additionally, the technical difficulty of the implementation stage was underestimated. At the early stage of experimentation, the results could not be reproduced consistently even after random seeds had been set, which required extensive troubleshooting through further background research. This process extended the implementation period and delayed some subsequent tasks.

Another major deviation from the original plan was the evolution of the model itself. The initial expectation was to refine the interim report model within a relatively limited scope. However, during the later stages of the project, it was necessary to address the limitations of the serial design. As a result, the project developed from the earlier S3ViT model into the final DBCP model. Although this change increased the workload, it improved the quality of the work and led to stronger experimental results.

This experience shows that the original plan was useful as a guideline, but it was a bit optimistic in estimating the time required for implementation and refinement. In particular, insufficient time was assigned for unexpected technical issues and model refinement. However, all essential work was completed before the deadline by rescheduling the timetable. Therefore, the project demonstrates that while careful initial planning is important, flexibility and continuous adjustment are equally necessary in a research project.

If the project were to be planned again, more buffer time would be allocated to implementation model refinement. In addition, secondary goals such as optional extensions beyond the core dissertation requirements, would be scheduled more conservatively. Overall, although the original plan was not followed exactly, it still provided a valuable framework for organizing the project.

Chapter 6

Conclusion

According to the five key challenges in HSI classification mentioned in the introduction section: high dimensionality, mixed pixels, limited availability of labeled samples, feature dilution caused by deep network and noisy label, this work focus on solving the first four challenges. By isolating the noisy label factor, this work solves the remaining four challenges as follows:

- **High-Dimensionality:** Given that high dimensionality is one of the most prominent challenges in HSI classification, this research prioritizes dimensionality reduction to ensure computational efficiency. Three different methods were integrated into the DBCP architecture. Firstly, in the Pixelwise ViT branch, PCA is employed as a preliminary step. This classical yet effective technique reduces the redundant spectral bands into a compact set of principal components, which is a standard practice in HSI literature. It facilitates the subsequent capture of long-range dependencies by the Transformer encoder. Secondly, in the CNN branch, a 3D-CNN layer is used to perform non-linear dimensionality reduction. By employing a spatial kernel size of (1, 1), the model effectively compresses the spectral dimension while preserving the integrity of the spatial resolution. This allows for the extraction of discriminative spectral signatures before they are passed to deeper feature extraction modules. Thirdly, the Separate Groupwise Convolution (SGC) block further reduces the dimensionality by splitting the spectral bands into four parallel subgroups. By processing these groups independently before concatenation, the SGC block significantly reduces the total number of trainable parameters and the overall computational complexity, which enables the model to learn spectral-spatial information without the heavy overhead associated with standard 3D-CNN.
- **Mixed Pixels (Spectral Mixing):** In the DBCP architecture, this problem is addressed through the interaction between the CNN and ViT branches. In the CNN branch, the SGC block and its multi-scale kernels captures neighborhood spatial correlations. By analyzing the identity of surrounding pixels, the model can infer the most probable category of a mixed pixel based on its environmental context. In the Pixelwise ViT branch, a self-attention mechanism is employed to model global spectral dependencies. For a boundary pixel with ambiguous spectral signatures, the self-attention mechanism identifies its strongest correlations with surrounding clusters of pure pixels. By aggregating information from the most relevant spatial neighbors across the entire neighborhood block, the model performs a robust contextual inference based on its neighbor pixels. The dual-branch fusion ensures that even when spectral information is ambiguous, spatial context can provide corrective cues.
- **Limited Training Samples and Inter-class Imbalance:** To address the issue of imbalanced class distributions common in HSI datasets, the Squeeze-and-Excitation (SE)

Block was used. By dynamically recalibrating channelwise feature importance, the model focuses on the most discriminative spectral bands, thereby enhancing the representation of minority classes.

- **Vanishing Gradients and Feature Dilution:** As models grow in depth, they often suffer from vanishing gradients and the loss of low-level structural details. The Dual-Branch architecture was specifically designed to solve this by providing parallel pathways for raw data. This structure ensures that both spatial textures and global spectral context are preserved and fused at the terminal stage, effectively preventing the degradation found in deep serial networks.

In conclusion, this research successfully addresses the core challenges of high dimensionality, mixed pixels, limited samples, and feature dilution through the DBCP architecture. By using 3D-CNN for local spatial-spectral feature extraction with a Pixelwise ViT for global contextual modeling, the model achieves a robust representation of hyperspectral data. The integration of PCA and SGC blocks ensures computational efficiency, while the SE block and dual-branch parallel structure effectively mitigate the issues of class imbalance and information loss in deep network. Experimental results across multiple benchmark datasets demonstrate that this approach not only preserves the integrity of spectral signatures but also leverages spatial context to enhance the classification accuracy in complex remote sensing scenes.

6.1 Future Work

Although the proposed DBCP model achieves strong classification performance across multiple benchmark datasets, several directions remain for future research. First, the current framework mainly focuses on supervised classification under limited labeled samples, while its robustness under more realistic settings such as noisy labels, deserves further investigation. Second, although the dual-branch architecture effectively combines local spectral-spatial feature extraction with global contextual modeling, the overall computational cost could be further reduced by exploring more lightweight attention mechanisms or more efficient spectral grouping strategies. Third, integrating self-supervised or semi-supervised learning strategies may further improve the model’s ability to learn discriminative representations from limited training samples. These directions could further enhance both the practical applicability and the generalization capability of the proposed model.

References

- [1] Mary B Stuart, Andrew JS McGonigle, and Jon R Willmott. Hyperspectral imaging in environmental monitoring: A review of recent developments and technological advances in compact field deployable systems. *Sensors*, 19(14):3071, 2019.
- [2] Muhammad Jaleed Khan, Hamid Saeed Khan, Adeel Yousaf, Khurram Khurshid, and Asad Abbas. Modern trends in hyperspectral image analysis: A review. *IEEE Access*, 6:14118–14129, 2018.
- [3] Megandhren Govender, Kershani Chetty, and Hartley Bulcock. A review of hyperspectral remote sensing and its application in vegetation and water resource studies. *Water Sa*, 33(2):145–151, 2007.
- [4] Leandro D Medus, Mohamed Saban, Jose V Francés-Víllora, Manuel Bataller-Mompeán, and Alfredo Rosado-Muñoz. Hyperspectral image classification using CNN: Application to industrial food packaging. *Food Control*, 125:107962, 2021.
- [5] Bing Lu, Phuong D Dao, Jiangui Liu, Yuhong He, and Jiali Shang. Recent advances of hyperspectral imaging technology and applications in agriculture. *Remote Sensing*, 12(16):2659, 2020.
- [6] Oscar Carrasco, Richard B Gomez, Arun Chainani, and William E Roper. Hyperspectral imaging applied to medical diagnoses and food safety. In *Geo-Spatial and temporal image and data exploitation III*, volume 5097, pages 215–221. SPIE, 2003.
- [7] Jaana Kuula, Ilkka Pölönen, Hannu-Heikki Puupponen, Tuomas Selander, Tapani Reinikainen, Tapani Kalenius, and Heikki Saari. Using VIS/NIR and IR spectral cameras for detecting and separating crime scene details. In *Sensors, and Command, Control, Communications, and Intelligence (C3I) Technologies for Homeland Security and Homeland Defense XI*, volume 8359, pages 150–160. SPIE, 2012.
- [8] Agustin Lobo. Image segmentation and discriminant analysis for the identification of land cover units in ecology. *IEEE Transactions on Geoscience and Remote Sensing*, 35(5):1136–1145, 1997.
- [9] Tianming Zhan, Bo Song, Le Sun, Xiuping Jia, Minghua Wan, Guowei Yang, and Zebin Wu. TDSSC: A three-directions spectral–spatial convolution neural network for hyperspectral image change detection. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 14:377–388, 2020.
- [10] Lefei Zhang, Liangpei Zhang, Dacheng Tao, Xin Huang, and Bo Du. Hyperspectral remote sensing image subpixel target detection based on supervised metric learning. *IEEE Transactions on Geoscience and Remote Sensing*, 52(8):4955–4965, 2013.
- [11] Bing Zhang, Di Wu, Li Zhang, Quanjun Jiao, and Qingting Li. Application of hyperspectral remote sensing for environment monitoring in mining areas. *Environmental Earth Sciences*, 65(3):649–658, 2012.

- [12] Shutao Li, Weiwei Song, Leyuan Fang, Yushi Chen, Pedram Ghamisi, and Jon Atli Benediktsson. Deep learning for hyperspectral image classification: An overview. *IEEE Transactions on Geoscience and Remote Sensing*, 57(9):6690–6709, 2019.
- [13] Xiaofeng Zhao, Junyi Ma, Lei Wang, Zhili Zhang, Yao Ding, and Xiongwu Xiao. A review of hyperspectral image classification based on graph neural networks. *Artificial Intelligence Review*, 58(6):172, 2025.
- [14] Bilal Alhayani and Haci Ilhan. Hyper spectral image classification using dimensionality reduction techniques. *International Journal of Innovative Research in Electrical, Electronics, Instrumentation and Control Engineering*, 5(4):71–74, 2017.
- [15] Antonio Plaza, Pablo Martinez, Rosa Perez, and Javier Plaza. A new approach to mixed pixel classification of hyperspectral imagery based on extended morphological profiles. *Pattern Recognition*, 37(6):1097–1116, 2004.
- [16] Zilong Zhong, Jonathan Li, David A Clausi, and Alexander Wong. Generative adversarial networks and conditional random fields for hyperspectral image classification. *IEEE Transactions on Cybernetics*, 50(7):3318–3329, 2019.
- [17] Lokesh Borawar and Ravinder Kaur. ResNet: Solving vanishing gradient in deep networks. In *Proceedings of International Conference on Recent Trends in Computing: ICRTC 2022*, pages 235–247. Springer, 2023.
- [18] Junjun Jiang, Jiayi Ma, Zheng Wang, Chen Chen, and Xianming Liu. Hyperspectral Image Classification in the Presence of Noisy Labels. *IEEE Transactions on Geoscience and Remote Sensing*, 57(2):851–865, 2019.
- [19] Yushi Chen, Hanlu Jiang, Chunyang Li, Xiuping Jia, and Pedram Ghamisi. Deep feature extraction and classification of hyperspectral images based on convolutional neural networks. *IEEE Transactions on Geoscience and Remote Sensing*, 54(10):6232–6251, 2016.
- [20] Swalpa Kumar Roy, Gopal Krishna, Shiv Ram Dubey, and Bidyut B Chaudhuri. HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification. *IEEE Geoscience and Remote Sensing Letters*, 17(2):277–281, 2019.
- [21] Wei Li, Guodong Wu, Fan Zhang, and Qian Du. Hyperspectral image classification using deep pixel-pair features. *IEEE Transactions on Geoscience and Remote Sensing*, 55(2):844–853, 2016.
- [22] Saeed Ghaderizadeh, Dariush Abbasi-Moghadam, Alireza Sharifi, Na Zhao, and Aqil Tariq. Hyperspectral image classification using a hybrid 3D-2D convolutional neural networks. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 14:7570–7588, 2021.
- [23] Xiangrong Zhang, Shouwang Shang, Xu Tang, Jie Feng, and Licheng Jiao. Spectral partitioning residual network with spatial attention mechanism for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–14, 2021.
- [24] Zhaohui Xue, Yiyang Zhou, and Peijun Du. S3Net: Spectral–spatial Siamese network for few-shot hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–19, 2022.

- [25] Danfeng Hong, Zhu Han, Jing Yao, Lianru Gao, Bing Zhang, Antonio Plaza, and Jocelyn Chanussot. SpectralFormer: Rethinking hyperspectral image classification with transformers. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–15, 2021.
- [26] Muhammad Ahmad, Usman Ghous, Muhammad Usama, and Manuel Mazzara. WaveFormer: Spectral–spatial wavelet transformer for hyperspectral image classification. *IEEE Geoscience and Remote Sensing Letters*, 21:1–5, 2024.
- [27] Shaohui Mei, Chao Song, Mingyang Ma, and Fulin Xu. Hyperspectral image classification using group-aware hierarchical transformer. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–14, 2022.
- [28] Linus Scheibenreif, Michael Mommert, and Damian Borth. Masked vision transformers for hyperspectral image classification. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2166–2176, 2023.
- [29] Yichu Xu, Di Wang, Lefei Zhang, and Liangpei Zhang. Dual selective fusion transformer network for hyperspectral image classification. *Neural Networks*, 187:107311, 2025.
- [30] Ji He, Lina Zhao, Hongwei Yang, Mengmeng Zhang, and Wei Li. HSI-BERT: Hyperspectral image classification using the bidirectional encoder representation from transformers. *IEEE Transactions on Geoscience and Remote Sensing*, 58(1):165–178, 2019.
- [31] Haoyang Yu, Zhen Xu, Ke Zheng, Danfeng Hong, Hao Yang, and Meiping Song. MSTNet: A multilevel spectral–spatial transformer network for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–13, 2022.
- [32] Zhuoyi Zhao, Xiang Xu, Shutao Li, and Antonio Plaza. Hyperspectral image classification using groupwise separable convolutional vision transformer network. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–17, 2024.
- [33] Huaiping Yan, Erlei Zhang, Jun Wang, Chengcai Leng, Anup Basu, and JinYE Peng. Hybrid Conv-ViT network for hyperspectral image classification. *IEEE Geoscience and Remote Sensing Letters*, 20:1–5, 2023.
- [34] Sen Jia, Yifan Wang, Shuguo Jiang, and Ruyan He. A center-masked transformer for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–16, 2024.
- [35] Rui Xu, Xue-Mei Dong, Weijie Li, Jiangtao Peng, Weiwei Sun, and Yi Xu. DBCTNet: Double branch convolution-transformer network for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–15, 2024.
- [36] Junjie Zhang, Zhe Meng, Feng Zhao, Hanqiang Liu, and Zhenhui Chang. Convolution transformer mixer for hyperspectral image classification. *IEEE Geoscience and Remote Sensing Letters*, 19:1–5, 2022.
- [37] Le Sun, Guangrui Zhao, Yuhui Zheng, and Zebin Wu. Spectral–spatial feature tokenization transformer for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–14, 2022.

- [38] Bing Tu, Xiaolong Liao, Qianming Li, Yishu Peng, and Antonio Plaza. Local semantic feature aggregation-based transformer for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–15, 2022.
- [39] Swalpa Kumar Roy, Ankur Deria, Chiranjibi Shah, Juan M Haut, Qian Du, and Antonio Plaza. Spectral–spatial morphological attention transformer for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 61:1–15, 2023.
- [40] Zhiwen Zhang, Teng Li, Xuebin Tang, Xiang Hu, and Yuanxi Peng. CAEVT: Convolutional autoencoder meets lightweight vision transformer for hyperspectral image classification. *Sensors*, 22(10):3902, 2022.
- [41] Le Sun, Hang Zhang, Yuhui Zheng, Zebin Wu, Zhonglin Ye, and Haixing Zhao. Mass-former: Memory-augmented spectral-spatial transformer for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–15, 2024.
- [42] Zhenqiu Shu, Yuyang Wang, and Zhengtao Yu. Dual attention transformer network for hyperspectral image classification. *Engineering Applications of Artificial Intelligence*, 127:107351, 2024.
- [43] Hao Sun, Xiangtao Zheng, Xiaoqiang Lu, and Siyuan Wu. Spectral–spatial attention network for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 58(5):3232–3245, 2019.
- [44] Swalpa Kumar Roy, Suvojit Manna, Tiecheng Song, and Lorenzo Bruzzone. Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 59(9):7831–7843, 2020.
- [45] Xu Tang, Fanbo Meng, Xiangrong Zhang, Yiu-Ming Cheung, Jingjing Ma, Fang Liu, and Licheng Jiao. Hyperspectral image classification based on 3-D octave convolution with spatial–spectral attention network. *IEEE Transactions on Geoscience and Remote Sensing*, 59(3):2430–2447, 2020.
- [46] Jing Yao, Danfeng Hong, Chenyu Li, and Jocelyn Chanussot. Spectralmamba: Efficient mamba for hyperspectral image classification. *arXiv preprint arXiv:2404.08489*, 2024.
- [47] Yan He, Bing Tu, Puzhao Jiang, Bo Liu, Jun Li, and Antonio Plaza. IGroupSS-Mamba: Interval group spatial-spectral mamba for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 2024.
- [48] Yan He, Bing Tu, Bo Liu, Jun Li, and Antonio Plaza. 3DSS-Mamba: 3D-spectral-spatial mamba for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 2024.
- [49] Ying Li, Haokui Zhang, and Qiang Shen. Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network. *Remote Sensing*, 9(1):67, 2017.
- [50] Li Wang, Jiangtao Peng, and Weiwei Sun. Spatial–spectral squeeze-and-excitation residual network for hyperspectral image classification. *Remote Sensing*, 11(7):884, 2019.
- [51] Craig Rodarmel and Jie Shan. Principal component analysis for hyperspectral image classification. *Surveying and Land Information Science*, 62(2):115–122, 2002.

- [52] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [53] Xiaofei Yang, Weijia Cao, Yao Lu, and Yicong Zhou. Hyperspectral image transformer classification networks. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–15, 2022.
- [54] Uzair Aslam Bhatti, Mengxing Huang, Harold Neira-Molina, Shah Marjan, Mehmood Baryalai, Hao Tang, Guilu Wu, and Sibghat Ullah Bazai. MFFCG–Multi feature fusion for hyperspectral image classification using graph attention network. *Expert Systems with Applications*, 229:120496, 2023.
- [55] Weiwei Song, Shutao Li, Leyuan Fang, and Ting Lu. Hyperspectral image classification with deep feature fusion network. *IEEE Transactions on Geoscience and Remote Sensing*, 56(6):3173–3184, 2018.
- [56] Yao Ding, Zhili Zhang, Xiaofeng Zhao, Danfeng Hong, Wei Cai, Chengguo Yu, Nengjun Yang, and Weiwei Cai. Multi-feature fusion: Graph neural network and CNN combining for hyperspectral image classification. *Neurocomputing*, 501:246–257, 2022.
- [57] B Ayerdi M Graña, MA Veganzons. Hyperspectral Remote Sensing Scenes, 2021. https://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes. Accessed: 2026-3-26.
- [58] RSIDEA Survey and Image Data Engineering Analysis Group. WHU-Hi: UAV-borne hyperspectral and high spatial resolution (H2) benchmark datasets for crop precise classification. https://rsidea.whu.edu.cn/resource_WHUHi_sharing.htm. Accessed: 2026-3-23.
- [59] Hyungtae Lee and Heesung Kwon. Contextual deep CNN based hyperspectral classification. In *2016 IEEE international geoscience and remote sensing symposium (IGARSS)*, pages 3322–3325. IEEE, 2016.
- [60] Amina Ben Hamida, Alexandre Benoit, Patrick Lambert, and Chokri Ben Amar. 3-D deep learning approach for remote sensing image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 56(8):4420–4434, 2018.

Appendix A

Example: Source Code Samples

```
1 class PatchSet(Dataset):
2     """ Generate 3D patch from hyperspectral dataset """
3     def __init__(self, data, pca_data, gt, patch_size, is_pred=False):
4         """
5         Args:
6             data: 3D hyperspectral image
7             pca_data: 3D hyperspectral image after pca
8             gt: 2D array of labels
9             patch_size: int, size of the 3D patch
10            is_pred: bool, create data without label for prediction (
11            default False)
12            """
13            super(PatchSet, self).__init__()
14            self.is_pred = is_pred
15            self.patch_size = patch_size
16            p = self.patch_size // 2
17            self.data = np.pad(data, ((p,p),(p,p),(0,0)), 'constant',
18            constant_values = 0)
19            self.pca_data = np.pad(pca_data, ((p,p),(p,p),(0,0)), 'constant',
20            constant_values = 0)
21            if is_pred:
22                gt = np.ones_like(gt)
23                self.label = np.pad(gt, (p,p), 'constant', constant_values = 0)
24                x_pos, y_pos = np.nonzero(gt)
25                x_pos, y_pos = x_pos + p, y_pos + p    ##indices after padding
26                self.indices = np.array([(x,y) for x,y in zip(x_pos, y_pos)])
27            if not is_pred:
28                np.random.shuffle(self.indices)
29
30            def __len__(self):
31                return len(self.indices)
32
33            def __getitem__(self, i):
34                x, y = self.indices[i]
35                x1, y1 = x - self.patch_size // 2, y - self.patch_size // 2
36                x2, y2 = x1 + self.patch_size, y1 + self.patch_size
37                data = self.data[x1:x2, y1:y2]
38                pca_data = self.pca_data[x1:x2, y1:y2]
39                label = self.label[x, y]
40                data = np.asarray(data, dtype='float32').transpose((2, 0, 1))
41                pca_data = np.asarray(pca_data, dtype='float32').transpose((2, 0,
42                1))
43                label = np.asarray(label, dtype='int64')
44                data = torch.from_numpy(data)
45                pca_data = torch.from_numpy(pca_data)
46                label = torch.from_numpy(label)
```

```

43     if self.is_pred:
44         return data, pca_data
45     else: return data, pca_data, label

```

Listing A.1: Patch Segmentation Code Sample

```

1 class ChanLayerNorm(nn.Module):
2     def __init__(self, dim, eps=1e-5):
3         super().__init__()
4         self.eps = eps
5         self.g = nn.Parameter(torch.ones(1, dim, 1, 1))
6         self.b = nn.Parameter(torch.zeros(1, dim, 1, 1))
7
8     def forward(self, x):
9         var = torch.var(x, dim=1, unbiased=False, keepdim=True)
10        mean = torch.mean(x, dim=1, keepdim=True)
11        return (x - mean) / (var + self.eps).sqrt() * self.g + self.b

```

Listing A.2: Channel Layer Normalization Code Sample

```

1 class SELayer(nn.Module):
2     def __init__(self, channel, reduction):
3         super().__init__()
4         self.avg_pool = nn.AdaptiveAvgPool3d(1)
5         self.fc = nn.Sequential(
6             nn.Linear(channel, channel // reduction, bias=False),
7             nn.ReLU(inplace=True),
8             nn.Linear(channel // reduction, channel, bias=False),
9             nn.Sigmoid()
10        )
11
12    def forward(self, x):
13        b, c, d, h, w = x.size()
14        y = self.avg_pool(x).view(b, c)
15        y = self.fc(y).view(b, c, 1, 1, 1)
16        x = x * y
17        return x

```

Listing A.3: SE Block Code Sample

```

1 def applyPCA(X, numComponents=15):
2     newX = np.reshape(X, (-1, X.shape[2]))
3     pca = PCA(n_components=numComponents, whiten=True)    ##PCA and
normalization
4     newX = pca.fit_transform(newX)
5     newX = np.reshape(newX, (X.shape[0],X.shape[1], numComponents))
6     return newX

```

Listing A.4: PCA Code Sample

```

1 class PixelViT(nn.Module):
2     def __init__(self, bands):
3         super().__init__()
4         self.layernorm1 = nn.LayerNorm(bands)
5         encoder_layer = nn.TransformerEncoderLayer(
6             d_model=bands,
7             nhead=NHEADS,
8             dim_feedforward=FEEDFORWARD_FACTOR*bands,
9             batch_first=True,

```

```

10         dropout=0.4,
11     )
12     self.attn = nn.TransformerEncoder(encoder_layer, num_layers=
NUM_LAYERS)
13     self.layernorm2 = nn.LayerNorm(bands)
14
15     def forward(self, x):
16         x = self.layernorm1(x)
17         x = self.attn(x)
18         x = self.layernorm2(x)
19         out = x.mean(dim=1)
20         return out

```

Listing A.5: Pixelwise ViT Code Sample

```

1 class DBCP(nn.Module):
2     def __init__(self, in_chs, patch_size, class_nums):
3         super().__init__()
4         self.in_chs = in_chs
5         self.patch_size = patch_size
6         self.num_blocks = 4
7
8         self.channelnorm = ChanLayerNorm(in_chs)
9         self.avg_pool = nn.AdaptiveAvgPool3d(1)
10        self.layernorm = nn.LayerNorm(320)
11
12        self.conv1 = nn.Sequential(
13            nn.Conv2d(in_channels=in_chs, out_channels=256,
kernel_size=1),
14            nn.BatchNorm2d(256),
15            nn.ReLU(inplace=True))
16        self.conv2 = nn.Sequential(
17            nn.Conv3d(in_channels=1, out_channels=8, kernel_size=(17,
1, 1), stride=(4, 1, 1)),
18            nn.BatchNorm3d(8),
19            nn.ReLU(inplace=True))
20        self.conv3 = nn.Sequential(
21            nn.Conv3d(in_channels=8, out_channels=16, kernel_size=(7,
3, 3)),
22            nn.BatchNorm3d(16),
23            nn.ReLU(inplace=True))
24        self.conv4 = nn.Sequential(
25            nn.Conv3d(in_channels=16, out_channels=32, kernel_size
=(7, 3, 3)),
26            nn.BatchNorm3d(32),
27            nn.ReLU(inplace=True))
28        self.conv5 = nn.Sequential(
29            nn.Conv3d(in_channels=32, out_channels=64, kernel_size
=(3, 3, 3)),
30            nn.BatchNorm3d(64),
31            nn.ReLU(inplace=True))
32
33        self.se3 = SELayer(channel=64*self.num_blocks, reduction=16)
34        self.pixel_vit = PixelViT(bands=64)
35        self.fc = nn.Linear(320, class_nums)
36
37        def forward(self, X, Y):
38            # CNN Branch
39            x = self.channelnorm(X)

```



```

40     x = self.conv1(x)
41     x = x.unsqueeze(1)
42     x = self.conv2(x)
43     blocks = []
44     for i in range(self.num_blocks):
45         block = x[:, :, i*15:(i+1)*15, :, :]
46         block = self.conv3(block)
47         block = self.conv4(block)
48         block = self.conv5(block)
49         blocks.append(block)
50     merged = torch.cat(blocks, dim=1)
51     merged = self.se3(merged)
52     B, C, D, H, W = merged.shape
53     cnn = self.avg_pool(merged).view(B, C)
54
55     # ViT Branch
56     B, C, H, W = Y.shape
57     y = Y.reshape(B, C, -1).permute(0, 2, 1)
58     vit = self.pixel_vit(y)
59
60     # feature fusion
61     combined_features = torch.cat((cnn, vit), dim=1)
62     combined_features = self.layernorm(combined_features)
63     out = self.fc(combined_features)
64     return out

```

Listing A.6: Main Model Code Sample

```

1 def train(model, train_loader, val_loader, save_path, device):
2     loss_list = []
3     acc_list = []
4     val_acc_list = []
5     val_epoch_list = []
6
7     model.to(device)
8     optimizer = torch.optim.Adam(model.parameters(), LR, weight_decay=
WEIGHT_DECAY)
9     loss_func = nn.CrossEntropyLoss()
10    train_num = train_loader.dataset.__len__()
11    val_num = val_loader.dataset.__len__()
12
13    fig = plt.figure()
14    ax1 = fig.add_subplot(2, 1, 1)
15    ax2 = fig.add_subplot(2, 1, 2)
16    try:
17        for e in range(EPOCH):
18            model.train()
19            avg_loss = 0.
20            train_acc = 0
21            for batch_idx, (data, pca_data, target) in enumerate(
train_loader):
22                data, pca_data, target = data.to(device), pca_data.to(device),
target.to(device)
23                optimizer.zero_grad()
24                out = model(data, pca_data)
25                target = target - 1  ## class 0 in out is class 1 in target
26                loss = loss_func(out, target)
27                loss.backward()
28                optimizer.step()

```

```

29         avg_loss += loss.item()
30         _,pred = torch.max(out,dim=1)
31         train_acc += (pred == target).sum().item()
32         loss_list.append(avg_loss/train_num)
33         acc_list.append(train_acc/train_num)
34         print(f"epoch {e}/{EPOCH} loss:{loss_list[e]}  acc:{acc_list[e]
35 }}")
36         ## valuation
37         if (e+1)%VAL_EPOCH == 0 or (e+1)==EPOCH:
38             val_acc =0
39             model.eval()
40             for batch_idx, (data, pca_data, target) in enumerate(
41 val_loader):
42                 data,pca_data,target = data.to(device),pca_data.to(
43 device),target.to(device)
44                 out = model(data,pca_data)
45                 target = target - 1  ## class 0 in out is class 1 in
46 target
47                 _,pred = torch.max(out,dim=1)
48                 val_acc += (pred == target).sum().item()
49                 val_acc_list.append(val_acc/val_num)
50                 val_epoch_list.append(e)
51                 print(f"epoch {e}/{EPOCH}  val_acc:{val_acc_list[-1]}")
52                 save_name = os.path.join(save_path, f"epoch_{e}_acc_{
53 val_acc_list[-1]:.4f}.pth")
54                 torch.save(model.state_dict(),save_name)
55             ax1.plot(np.arange(e+1),loss_list)
56             ax1.set_title('loss')
57             ax1.set_xlabel('epoch')
58             ax2.plot(np.arange(e+1),acc_list,label = 'train_acc')
59             ax2.plot(val_epoch_list,val_acc_list,label = 'val_acc')
60             ax2.set_title('acc')
61             ax2.set_xlabel('epoch')
62             ax2.legend()
63         except Exception as exc:
64             print(exc)
65         finally:
66             print(f'Stop in epoch {e}')
67
68     return val_acc_list, val_epoch_list

```

Listing A.7: Training Code Sample

```

1 def test(best_model, label, all_loader, test_gt, run_idx, device):
2     best_model.to(device)
3     best_model.eval()
4     pred_map = []
5     for batch_idx, (data, pca_data) in enumerate(all_loader):
6         data = data.to(device)
7         pca_data = pca_data.to(device)
8         target = best_model(data, pca_data)
9         _, pred = torch.max(target, dim=1)
10        pred_map += [np.array(pred.detach().cpu() + 1)]  ## class 0 in
11        pred_map is class 1 in gt
12        pred_map = np.asarray(np.hstack(pred_map), dtype=np.uint8).reshape(
13        label.shape[0], label.shape[1])
14
15        spectral.save_rgb(os.path.join(SAVE_PATH,f"prediction_run{run_idx}.jpg"
16        ),pred_map*(label!=0),colors = spectral.spy_colors)

```

```
14     spectral.imshow(classes=pred_map*(label!=0), title='prediction',  
15     figsize=(7, 7))  
16     plt.show()  
17  
18     test_pred = pred_map[test_gt != 0]  
19     test_true = test_gt[test_gt != 0]  
20  
21     return test_pred, test_true
```

Listing A.8: Testing Code Sample